Perspt Documentation

Release 0.6.0

Ronak Rathore, Vikrant Rathore

May 25, 2026

Getting Started

Your Terminal's Window to the AI World

Perspt is a high-performance terminal-based LLM interface that serves two purposes: a **simple CLI for testing and comparing LLM services** across 8 providers, and an **experimental implementation** of the **SRBN (Stabilized Recursive Barrier Network)** engine from the paper *"Stability is All You Need: Lyapunov-Guided Hierarchies for Long-Horizon LLM Reliability"* by **Vikrant R. and Ronak R.** (pre-publication). The SRBN agent plans multi-file projects as directed acyclic graphs, verifies each node with real LSP diagnostics and tests, and commits only when Lyapunov energy converges. The theoretical framework is mature; the implementation is under active development.

Chapter 1

Introduction to Perspt

1.1 What is Perspt?

Perspt (pronounced "perspect," short for **Per**sonal **S**pectrum **P**ertaining **T**houghts) is a high-performance, terminal-based interface to Large Language Models. It serves two complementary purposes:

1. **A simple CLI for testing LLM services** — Connect to OpenAI, Anthropic, Google Gemini, Groq, Cohere, xAI, DeepSeek, or Ollama with a single command. Auto-detect your API key, chat interactively in a beautiful TUI, or pipe responses through the simple-chat mode. Perspt makes it effortless to evaluate and compare different LLM providers from your terminal.
2. **An experimental implementation of the SRBN engine** — Perspt's agent mode is a practical implementation of the **Stabilized Recursive Barrier Network (SRBN)** framework described in the paper *"Stability is All You Need: Lyapunov-Guided Hierarchies for Long-Horizon LLM Reliability"* by **Vikrant R. and Ronak R.** (pre-publication). The SRBN engine decomposes coding tasks into DAGs, uses Lyapunov energy as a stability measure through multi-stage verification barriers, and commits only when energy converges — applying control-theoretic ideas to autonomous code generation. The theoretical framework is mature; the implementation is under active development and has not yet been benchmarked.

Version 0.6.0 "kukuza" Highlights

Ecosystem & Dependency Modernization:

- **Major Upgrades** — Bumped core dependencies to their latest major versions including `duckdb (=1.10503.1)`, `starlark (0.14)`, `genai (0.6.1)`, `askama (0.16)`, and `diffy (0.5)`.
- **Enterprise AI Providers** — Integrated native support for **AWS Bedrock** and **Google Agent Platform** (formerly Vertex AI) with advanced IAM credentials and OAuth2 token authorization.
- **State-of-the-Art Models** — Added full, out-of-the-box support for the newest generation of models, including `gpt-5.5` and `claude-sonnet-4.7`.

Version 0.5.9 "□□□□" Highlights

Robust Correction Loop Contracts (PSP-7):

- **Structured Artifact Bundle Format** — Switched correction prompt from free-form output to a strict JSON schema, ensuring the LLM explicitly declares target paths and artifacts.
- **Typed Parse Pipeline** — Replaced Option-based extraction with a 5-layer fail-closed parse pipeline that classifies retries (Retarget, MalformedRetry, SupportFileViolation, Replan) for intelligent convergence.

- **Manifest Policy Enforcement** — Added semantic validation to prevent implicit mutation of root manifests unless explicitly requested.
- **Strict Budget Exhaustion** — Upgraded budget checks to respect step and revision caps alongside cost, preventing runaway loops.

1.2 Architecture

Perspt is built as a **9-crate Rust workspace**:

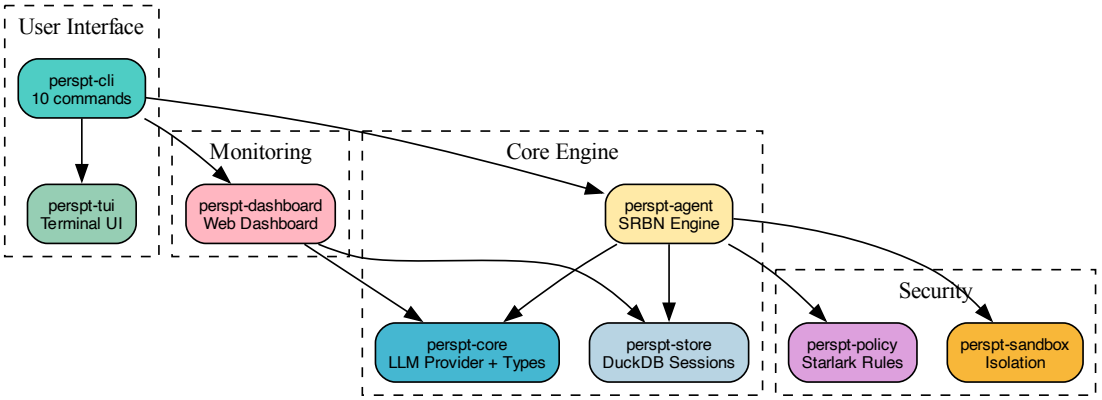


Figure 1: Perspt Architecture Overview

1.3 Key Features

SRB	**Agent Mode**	Experimental autonomous multi-file coding. Plans a task DAG, generates artifact bundles per node, verifies with LSP + tests, retries until energy converges, and commits to the ledger.
LLM	**Multi-Provider**	OpenAI, Anthropic, Google Gemini, Groq, Cohere, XAI, DeepSeek, and Ollama (local). Zero-config auto-detection from environment variables.
LSP	**Sensor Architecture**	Plugin-driven LSP selection: `rust-analyzer` for Rust, `ty` or `pyright` for Python, `typescript-language-server` for JS/TS, `gopls` for Go. Computes V_syn.
Test	**Test Runner**	pytest integration with weighted failure scoring. Critical tests carry weight 10, high-priority 3, low-priority 1. Produces V_log.
Led	**Merkle Ledger**	DuckDB-backed cryptographic change tracking. Supports rollback, session resume, energy history, and escalation reports.
Pol-icy	**Security**	Starlark policy engine validates commands before execution. Workspace-bound enforcement prevents escaping the project directory.
Bud-get	**Token Budget**	Per-session cost tracking with `--max-cost` USD limit and `--max-steps` iteration cap.
TUI	**Terminal UI**	Ratatui-based with markdown rendering, diff viewer, task tree, dashboard, review modal, and logs viewer.
Web	**Dashboard**	Browser-based real-time monitoring of agent execution, energy, LLM telemetry, sandbox branches, and decision traces.

1.4 SRBN: Stabilized Recursive Barrier Network

The SRBN engine in Perspt is based on the paper *"Stability is All You Need: Lyapunov-Guided Hierarchies for Long-Horizon LLM Reliability"* by **Vikrant R. and Ronak R.** (pre-publication). The paper introduces a topological framework that reformulates LLM agency as a sheaf-theoretic control problem, replacing probabilistic search with Lyapunov stability analysis. Key theoretical contributions include:

- **Input-to-State Stability (ISS)** proof showing bounded reasoning errors result in bounded system deviation (paper result)
- **Flow Matching Barriers** that project diverging agent trajectories back onto the safe manifold (paper result)
- **Adaptive Flow Speculation** for latency reduction via branch prediction
- Theoretical reliability scaling from exponential decay to logarithmic: $O(\log N)$ (paper prediction)

Perspt implements this theory as an experimental coding agent, governed by PSP-7. The mathematical framework is mature; empirical benchmarks on this implementation have not yet been published.

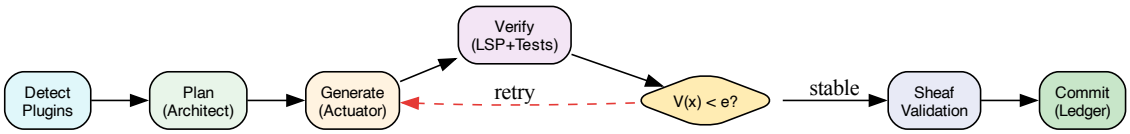


Figure 2: SRBN Control Flow (PSP-5)

Lyapunov Energy:

$$V(x) = \alpha \cdot V_{syn} + \beta \cdot V_{str} + \gamma \cdot V_{log} + V_{boot} + V_{sheaf}$$

- **V_syn** — LSP diagnostic count (errors + warnings)
- **V_str** — Structural contract violations
- **V_log** — Weighted test failures (pytest)
- **V_boot** — Bootstrap command exit codes (build, init)
- **V_sheaf** — Cross-node consistency failures

Default weights: alpha = 1.0, beta = 0.5, gamma = 2.0. Convergence threshold: epsilon = 0.10.

1.5 CLI Commands

Command	Description	Example
`chat`	Interactive TUI chat (default)	`perspt chat --model gemini-pro-latest`
`agent`	Autonomous multi-file coding (experimental)	`perspt agent "Create a REST API in Rust"`
`init`	Initialize project config	`perspt init --memory --rules`
`config`	View or edit configuration	`perspt config --show`
`ledger`	Query Merkle change history	`perspt ledger --recent`
`status`	Session lifecycle and energy	`perspt status`
`abort`	Cancel running session	`perspt abort`
`resume`	Resume interrupted session	`perspt resume --last`
`logs`	View LLM request logs	`perspt logs --tui`
`dashboard`	Real-time web monitoring	`perspt dashboard`
`simple-chat`	CLI chat without TUI	`perspt simple-chat`

1.6 Supported Providers

Provider	Environment Variable	Notes
OpenAI	`OPENAI_API_KEY`	GPT-4o, o3-mini, o1-preview, GPT-4.1
Anthropic	`ANTHROPIC_API_KEY`	Claude Sonnet 4, Claude Opus 4
Google	`GEMINI_API_KEY`	Gemini 2.5 Pro/Flash, Gemini 3.1 Pro/Flash
Groq	`GROQ_API_KEY`	Llama-based models (ultra-fast inference)
Cohere	`COHERE_API_KEY`	Command R+
XAI	`XAI_API_KEY`	Grok
DeepSeek	`DEEPSEEK_API_KEY`	DeepSeek Coder
Ollama	*(none)*	Local models — no API key required

1.7 Philosophy

> *The keyboard hums, the screen aglow,*
> *AI's wisdom, a steady flow.*
> *Through SRBN's loop, stability we find,*
> *Code that works, refined and aligned.*
>
> —The Perspt Manifesto

Perspt embodies the belief that AI tools should be:

- **Accessible** — A simple `perspt` command connects you to any LLM provider
- **Fast** — Rust-native performance with async streaming
- **Stable** — Lyapunov energy guides convergence before commit (SRBN agent, based on paper theory)
- **Secure** — Policy-controlled execution with workspace bounds
- **Extensible** — Modular 9-crate architecture
- **Experimental** — A testbed for control-theoretic approaches to LLM reliability

1.8 Next Steps

Quick Start Get running in 5 minutes.

Quick Start Agent Mode Autonomous multi-file coding.

Agent Mode Tutorial Architecture Understand the 9-crate design.

Architecture

Chapter 2

Quick Start

Get Perspt running in 5 minutes.

2.1 Prerequisites

Rust 1.82+	rustup.rs for building from source
API Key	From any provider (OpenAI, Anthropic, Google, etc.) OR Ollama for local models

2.2 Installation

From Source (Recommended)

```
git clone https://github.com/eonseed/perspt.git
cd perspt
cargo build --release
```

Cargo Install

```
cargo install perspt
```

With Ollama (No API Key)

```
# Install Ollama
brew install ollama  # macOS

# Start and pull a model
ollama serve
ollama pull llama3.2
```

2.3 Set Your API Key

```
# Choose your provider
export OPENAI_API_KEY="sk-..."        # OpenAI
export ANTHROPIC_API_KEY="sk-ant-..." # Anthropic
export GEMINI_API_KEY="..."           # Google Gemini
```

2.4 Run Your First Chat

```
# Start the TUI (auto-detects provider from env)
./target/release/perspt

# Or with a specific model
perspt chat --model gemini-pro-latest
```

Type your message and press **Enter**. Press **Esc** to exit.

2.5 Try Agent Mode

Let the experimental SRBN agent autonomously plan and build multi-file projects:

```
# Create a project in a new directory
perspt agent -w ./my-calculator "Create a Python calculator package with
add, subtract, multiply, divide. Include type hints and pytest tests."

# Auto-approve all changes (headless)
perspt agent -y -w ./my-api "Build a REST API in Rust with Axum"

# Use specific models per tier
perspt agent \
  --architect-model gemini-pro-latest \
  --actuator-model gemini-3.1-flash-lite-preview \
  -w ./project "Create an ETL pipeline in Python"
```

The SRBN engine will:

1. **Detect** — Identify language plugins and workspace state
2. **Plan** — Architect decomposes task into a DAG with ownership closure
3. **Generate** — Actuator emits multi-file artifact bundles per node
4. **Verify** — LSP diagnostics + tests compute Lyapunov energy V(x)
5. **Converge** — Retry with grounded error feedback until V(x) < epsilon
6. **Sheaf Check** — Validate cross-node consistency
7. **Commit** — Record stable state in Merkle ledger

See also

Agent Mode Tutorial for a full walkthrough.

2.6 Choose Your Mode

Mode	Command	Best For
Chat TUI	`perspt` or `perspt chat`	Interactive conversations with markdown rendering
Agent	`perspt agent "<task>"`	Autonomous multi-file code generation (experimental)
Simple Chat	`perspt simple-chat`	Scripting, pipelines, no TUI
Status	`perspt status`	Check current agent session

2.7 Essential Commands

```
# Configuration
perspt config --show           # View current config
perspt config --edit           # Edit in $EDITOR
perspt init --memory --rules   # Initialize project

# Agent management
perspt status                  # Current session status
perspt abort                   # Cancel current session
perspt resume --last           # Resume last interrupted session

# Change tracking
perspt ledger --recent         # View recent changes
perspt ledger --rollback abc   # Rollback to commit
perspt ledger --stats          # Session statistics

# Debugging
perspt logs --tui              # Interactive log viewer
perspt logs --last             # Most recent session
perspt logs --stats            # Usage statistics
```

2.8 Key Bindings (Chat TUI)

Key	Action
Enter	Send message
Esc	Exit application
Up / Down	Scroll chat history
Page Up / Down	Fast scroll
/save	Save conversation (command)

2.9 Next Steps

Tutorials Step-by-step learning guides.

Tutorials Configuration Customize providers and models.

Configuration Agent Deep Dive Master autonomous coding.

Agent Mode Tutorial Architecture Understand the 9-crate design.

Architecture

Chapter 3

Installation

3.1 System Requirements

- **OS**: macOS, Linux, Windows (WSL recommended)
- **Rust**: 1.75+ (for building from source)
- **Terminal**: 256-color support recommended

3.2 Install from Source

```
# Clone the repository
git clone https://github.com/eonseed/perspt.git
cd perspt

# Build in release mode
cargo build --release

# The binary is at target/release/perspt
# Optionally, copy to your PATH:
cp target/release/perspt ~/.local/bin/
```

3.3 Install with Cargo

```
cargo install --path .
```

3.4 Verify Installation

```
perspt --version
# perspt 0.5.6

perspt --help
```

3.5 Provider Setup

Set at least one API key:

Gemini (recommended)

```
export GEMINI_API_KEY="your-key"
```

OpenAI

```
export OPENAI_API_KEY="sk-xxx"
```

Anthropic

```
export ANTHROPIC_API_KEY="sk-ant-xxx"
```

Ollama (local)

```
# No API key needed
ollama serve
ollama pull llama3.2
```

See *Providers* for all supported providers.

3.6 Agent Mode Prerequisites

For agent mode, install the tool binaries for your target language:

Python

```
# uv (package manager + project init)
curl -LsSf https://astral.sh/uv/install.sh | sh

# ty (type checker / LSP)
pip install ty

# pytest (test runner)
pip install pytest
```

Rust

```
# Rust toolchain
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh

# rust-analyzer (LSP)
rustup component add rust-analyzer
```

JavaScript

```
# Node.js and npm
# (install via your package manager)

# TypeScript (LSP)
npm install -g typescript
```

3.7 Optional: Documentation Build

To build the documentation locally:

```
# Install uv (Python package manager)
curl -LsSf https://astral.sh/uv/install.sh | sh

# Build HTML docs
cd docs/perspt_book && uv run make html

# Open in browser
open docs/perspt_book/build/html/index.html
```

Chapter 4

Getting Started

This guide walks through installation, first chat, and first agent run.

4.1 Prerequisites

4.1.1 System Requirements

Component	Requirement
Operating System	Linux, macOS, or Windows
Rust Toolchain	Rust 1.82.0 or later
Terminal	Any modern terminal emulator with UTF-8 support
Network	Required for cloud LLM API calls (not needed for Ollama)

4.1.2 API Keys

You need an API key from at least one provider. Set it as an environment variable and Perspt will auto-detect the provider:

```
# Pick one — Perspt auto-detects from the environment
export OPENAI_API_KEY="sk-..."
export ANTHROPIC_API_KEY="sk-ant-..."
export GEMINI_API_KEY="..."

# Ollama needs no key — just run: ollama serve
```

> **Note**
>
> **Auto-detection priority**: OpenAI > Anthropic > Gemini > Groq > Cohere > XAI > DeepSeek > Ollama.

4.2 Quick Installation

From Source (Recommended)

```
git clone https://github.com/eonseed/perspt.git
cd perspt
cargo build --release
./target/release/perspt --version
```

Cargo Install

```
cargo install perspt
perspt --version
```

Binary Download

```
curl -L https://github.com/eonseed/perspt/releases/latest/download/perspt-linux-x86_64.tar.
↪gz | tar xz
chmod +x perspt && sudo mv perspt /usr/local/bin/
```

4.3 Your First Chat

TUI mode (default) provides a rich terminal interface with markdown rendering:

```
export GEMINI_API_KEY="your-key"
perspt
```

You will see a chat interface. Type a message and press **Enter**. Perspt streams the response in real time with markdown formatting. Press **Esc** to exit.

Simple CLI mode is a minimal text interface suitable for scripting:

```
perspt simple-chat
# Or with logging
perspt simple-chat --log-file session.txt
```

Type your question, get a streamed answer. Type `exit` or press `Ctrl+D` to quit.

4.4 Your First Agent Run

Agent mode (experimental) lets the SRBN engine plan and write multi-file projects autonomously:

```
# Create a new Python package
perspt agent -w ./demo-calc \
  "Create a Python calculator package with add, subtract, multiply, divide.
   Include type hints, a pyproject.toml, and pytest tests."
```

What happens:

1. **Detection** — Perspt identifies Python from the task description, selects the `python` plugin (`ty` LSP, `pytest` runner, `uv init --lib`).
2. **Planning** — The Architect model decomposes the task into a DAG of nodes, each owning specific output files (ownership closure rule).
3. **Execution** — Nodes execute in topological order. For each node, the Actuator generates a multi-file artifact bundle (writes, diffs, commands).

4. **Verification** — LSP diagnostics compute V_syn, `pytest` computes V_log, and bootstrap commands compute V_boot. Total energy V(x) is checked.
5. **Review** — In interactive mode, a diff viewer presents changes for approval. In headless mode (`--yes`), all changes are auto-approved.
6. **Commit** — Stable nodes are recorded in the Merkle ledger.

After completion, inspect the output:

```
ls demo-calc/
# pyproject.toml  src/  tests/  uv.lock

cd demo-calc && uv run pytest -v
```

4.4.1 Headless Mode

For CI/CD or batch workflows, use `--yes` to skip all interactive prompts:

```
perspt agent --yes -w ./output "Build a Rust CLI that converts CSV to JSON"
```

Combine with `--defer-tests` for faster iteration (skips V_log during coding, only runs tests at the final sheaf validation stage):

```
perspt agent --yes --defer-tests -w ./output "Build a data pipeline"
```

4.5 Switching Models

```
# Chat with a specific model
perspt chat --model gemini-pro-latest

# Agent with per-tier models
perspt agent \
  --architect-model gemini-pro-latest \
  --actuator-model gemini-3.1-flash-lite-preview \
  --verifier-model gemini-pro-latest \
  --speculator-model gemini-3.1-flash-lite-preview \
  "Build a web scraper"
```

4.6 Next Steps

Tutorials Step-by-step learning path.

Tutorials Configuration Providers, models, and preferences.

Configuration Agent Mode Full SRBN agent walkthrough.

Agent Mode Tutorial Concepts SRBN architecture and energy model.

Concepts

Chapter 5

Tutorials

Step-by-step guides to master Perspt.

First Chat Your first conversation with an LLM.

First Chat Agent Mode Autonomous multi-file coding with the experimental SRBN engine.

Agent Mode Tutorial Headless Mode Fully autonomous operation for CI/CD and batch workflows (experimental).

Headless Mode Local Models Use Ollama for offline AI.

Local Models with Ollama Custom Workflows Scripting, batch processing, and CI/CD integration.

Custom Workflows Example: Python ETL Build a data pipeline with agent mode.

Example: Python ETL Pipeline Example: Rust CLI Build a Rust command-line tool.

Example: Rust CLI Tool Example: Scientific Computing CFD simulation setup and wind tunnel analysis.

Example: Scientific Computing Dashboard Monitoring Watch agent execution in the browser.

Monitoring Agent Execution with the Dashboard

5.1 Learning Path

Order	Tutorial	Outcome
1	*First Chat*	Understand basic TUI interaction
2	*Local Models with Ollama*	Set up Ollama for privacy
3	*Agent Mode Tutorial*	Master multi-file autonomous coding
4	*Headless Mode*	Run agent without interactive prompts
5	*Custom Workflows*	Automate with scripts and CI/CD
6	*Example: Python ETL Pipeline*	Build a real Python project from scratch
7	*Example: Rust CLI Tool*	Build a real Rust project from scratch
8	*Example: Scientific Computing*	Apply agent mode to scientific computing
9	*Monitoring Agent Execution with the Dashboard*	Monitor agent sessions via the web dashboard

5.1.1 First Chat

Start your first conversation with an LLM using Perspt.

Prerequisites

- Perspt installed (see *Installation*)
- An API key from any provider, or Ollama running locally

Step 1: Set Your API Key

```
export GEMINI_API_KEY="your-key"
```

Step 2: Launch the TUI

```
perspt
```

Perspt auto-detects the provider from the environment variable and launches the chat TUI. You will see a status bar showing the provider and model.

Step 3: Chat

Type a message and press **Enter**. Perspt streams the response in real time with markdown formatting (code blocks, headers, lists, bold, italic).

```
You: Explain Rust's ownership model in 3 sentences.

Assistant: Rust's ownership model ensures each value has exactly one owner at a
time. When the owner goes out of scope, the value is automatically dropped.
Borrowing rules allow temporary references without transferring ownership, and
the compiler enforces these rules at compile time to prevent data races and
dangling pointers.
```

Step 4: Navigate

Key	Action
Enter	Send message
Ctrl+J	Insert newline in input
Page Up / Down	Scroll chat history
Ctrl+Up / Down	Scroll one line
Esc or **Ctrl+Q**	Exit
/save	Save conversation to file

Step 5: Try Simple CLI Mode

For a minimal interface without the TUI:

```
perspt simple-chat
# Or with logging:
perspt simple-chat --log-file session.txt
```

Type your question, get a streamed text answer. Type `exit` or `Ctrl+D` to quit.

Next Steps

- *Agent Mode Tutorial* — Try autonomous multi-file coding
- *Local Models with Ollama* — Use Ollama for offline conversations

5.1.2 Agent Mode Tutorial

Master autonomous multi-file code generation with the experimental SRBN engine.

Overview

Agent mode lets Perspt plan, write, test, and commit multi-file projects autonomously. The PSP-5 runtime decomposes tasks into a directed acyclic graph (DAG) of nodes, each owning specific output files, verified by real LSP diagnostics and test runners.

> **Experimental Feature**
>
> Agent mode implements the SRBN theoretical framework. The engine is functional and usable, but has not yet been benchmarked. Results may vary depending on model capability and task complexity.

Prerequisites

- Perspt v0.5.6+
- An API key for a capable model
- For Python projects: `uv` and `python3` installed
- For Rust projects: `cargo` and `rustc` installed

Basic Usage

```
# Plan and build a project in a new directory
perspt agent -w ./my-project "Create a Python calculator package"

# Auto-approve all changes (headless)
perspt agent -y -w ./my-project "Create a REST API in Rust"

# Use specific models per tier
perspt agent \
  --architect-model gemini-pro-latest \
  --actuator-model gemini-3.1-flash-lite-preview \
  -w ./project "Build an ETL pipeline"
```

Step-by-Step: Python Calculator

Step 1: Start the Agent

```
mkdir calc-demo && cd calc-demo
perspt agent -w . \
  "Create a Python calculator package with add, subtract, multiply,
   divide operations. Include type hints, a pyproject.toml with
   build-system, and comprehensive pytest tests."
```

Step 2: Watch the SRBN Loop

The agent proceeds through the PSP-5 phases:

Detection — Perspt identifies Python from the task description and selects the `python` plugin:

```
[detect] Workspace: greenfield
[detect] Plugin: python (LSP: ty, tests: pytest, init: uv init --lib)
```

Planning — The Architect decomposes the task into a DAG:

```
[plan] TaskPlan: 4 nodes, 1 plugin
[plan]   node-1: Initialize Python package (Command)
[plan]   node-2: Create calculator module (Code) -> [node-1]
[plan]   node-3: Create main entry point (Code) -> [node-2]
[plan]   node-4: Write pytest tests (UnitTest) -> [node-2]
```

Each node owns specific output files (ownership closure):

- node-1: `pyproject.toml`
- node-2: `src/calculator/__init__.py`, `src/calculator/core.py`
- node-3: `src/main.py`
- node-4: `tests/test_calculator.py`

Execution — Nodes execute in topological order. For each node:

1. Actuator generates a multi-artifact bundle (writes, diffs, commands)
2. Files are applied transactionally
3. LSP diagnostics run (V_syn), contracts check (V_str), tests run (V_log)
4. Bootstrap commands check (V_boot)

```
[node-2] Generating artifact bundle...
[node-2] Bundle: 2 files created (write), 0 modified (diff)
[node-2] Verification:
          V_syn  = 0.00 (ty: 0 errors, 0 warnings)
          V_str  = 0.00 (contracts satisfied)
          V_log  = 0.00 (deferred or passed)
          V_boot = 0.00 (all commands succeeded)
          V_sheaf = 0.00 (pending final check)
          V(x)  = 0.00 < epsilon=0.10 -> STABLE
```

Step 3: Review Changes

In interactive mode, the review modal presents grouped diffs:

```
Review Node 2: Create calculator module
────────────────────────────────────────
Bundle: 2 created, 0 modified
+ src/calculator/__init__.py  [create] (3 lines)
+ src/calculator/core.py      [create] (45 lines)

Verification: V_syn OK | V_str OK | V_log OK | V_boot OK
Energy: V(x) = 0.00

[y] Approve  [n] Reject  [c] Correct  [e] Edit  [d] Diff
```

Actions:

- **y** — Approve and commit to ledger
- **n** — Reject and re-generate from scratch
- **c** — Send correction feedback to the agent
- **e** — Open files in your editor, then return
- **d** — Toggle full unified diff view

Step 4: Inspect Results

After all nodes converge and pass sheaf validation:

```
ls -la
# pyproject.toml  src/  tests/  uv.lock

# Run the tests
cd calc-demo && uv run pytest -v

# Check the ledger
perspt ledger --recent
```

Model Tier Configuration

Assign specialized models to each SRBN phase:

```
perspt agent \
  --architect-model gemini-pro-latest \
  --actuator-model gemini-3.1-flash-lite-preview \
  --verifier-model gemini-pro-latest \
  --speculator-model gemini-3.1-flash-lite-preview \
  -w ./project "Build a web server"
```

Tier	Purpose	Recommendation
Architect	Task decomposition, DAG planning	Deep reasoning model (e.g., Gemini Pro, Claude Sonnet)
Actuator	Code generation, artifact bundles	Fast coding model (e.g., Gemini Flash)
Verifier	Stability analysis, contract checking	Analytical model (e.g., Gemini Pro)
Speculator	Branch prediction, lookahead	Ultra-fast model (e.g., Gemini Flash Lite)

Energy Tuning

Customize the Lyapunov energy weights:

```
# Prioritize test passing (higher gamma)
perspt agent --energy-weights "1.0,0.5,3.0" -w . "Add tests"

# Prioritize type safety (higher alpha)
perspt agent --energy-weights "2.0,0.5,1.0" -w . "Add type hints"

# Custom convergence threshold
perspt agent --stability-threshold 0.5 -w . "Quick prototype"
```

Execution Modes

Mode	Behavior
`cautious`	Prompt for approval on every node
`balanced`	Prompt when complexity > K (default)
`yolo`	Auto-approve everything (use with caution)

```
perspt agent --mode cautious -w . "Modify database schema"
```

Cost and Step Limits

```
# Maximum $5 cost
perspt agent --max-cost 5.0 -w . "Large refactor"

# Maximum 20 iterations across all nodes
perspt agent --max-steps 20 -w . "Iterative improvement"
```

Managing Sessions

```
# Show session status: lifecycle counts, energy breakdown, escalations
perspt status

# Abort the current session
perspt abort

# Resume the last interrupted session with trust context
perspt resume --last
```

The `status` command shows per-node lifecycle counts (queued, running, verifying, retrying, completed, failed, escalated), the latest energy breakdown, total retry count, and recent escalation reports.

The `resume` command displays trust context before resuming: escalation count, last energy state, and total retries across all nodes.

LLM Logging

```
# Log all LLM calls to the DuckDB store
perspt agent --log-llm -w . "Debug task"

# View logs interactively
perspt logs --tui

# View most recent session
perspt logs --last

# Usage statistics
perspt logs --stats
```

Best Practices

1. **Start with a clear task description** — Include language, package structure, and testing requirements in the prompt
2. **Use workspace directories** — Always specify `-w <dir>` for clarity
3. **Set cost limits** — Use `--max-cost` to prevent runaway spending
4. **Review before committing** — In interactive mode, inspect diffs carefully
5. **Use per-tier models** — Match model capabilities to task complexity
6. **Track changes** — Use `perspt ledger` to review and rollback

Troubleshooting

Agent stuck in retry loop:

- Check LSP is working: `ty check file.py` or `cargo check`
- Lower stability threshold: `--stability-threshold 0.5`
- Use `--defer-tests` to skip V_log during coding

High energy despite clean code:

- Check test failures: `uv run pytest -v`
- Review LSP diagnostics
- Adjust weights: `--energy-weights "0.5,0.5,1.0"`

Plugin not detected:

- Ensure required binaries are installed (`uv`, `cargo`, `node`, etc.)
- Check `perspt status` for active plugins

See Also

- *Headless Mode* — Fully autonomous operation
- *SRBN Architecture* — SRBN technical details
- *Agent Options Reference* — Full CLI reference

5.1.3 Headless Mode

Run Perspt's experimental SRBN agent without interactive prompts. This is designed for CI/CD pipelines, batch code generation, and automated workflows.

Overview

In headless mode (`--yes` flag), the agent auto-approves all changes, skipping the interactive review modal. Combined with `--workdir` and `--defer-tests`, it enables fully autonomous project generation.

```
perspt agent --yes -w /tmp/output "Create a Python ETL pipeline"
```

When to Use Headless Mode

- **CI/CD pipelines** — Generate boilerplate or scaffold projects in automation
- **Batch processing** — Run multiple agent tasks in sequence from a script
- **Rapid prototyping** — Skip review when iterating quickly
- **Testing the agent** — Validate agent behavior without manual intervention

When NOT to use headless mode:

- **Production codebases** — Always review changes before committing
- **Security-sensitive projects** — Manual review catches policy violations
- **Learning** — Interactive mode teaches you how SRBN works

Basic Headless Run

```
export GEMINI_API_KEY="your-key"

# Create a project autonomously
perspt agent --yes -w /tmp/my-project \
  "Create a Python data validation library using Pydantic.
   Include src layout, pyproject.toml, and pytest tests."
```

The agent will:

1. Detect language plugins
2. Plan the task DAG
3. Execute all nodes, auto-approving each
4. Run verification (LSP + tests) on each node
5. Commit stable nodes to the ledger
6. Print a summary

Key Flags

Flag	Description
`--yes` / `-y`	Auto-approve all changes (headless mode)
`-w, --workdir <DIR>`	Working directory for the project
`--defer-tests`	Skip V_log during node coding; only run tests at sheaf validation
`--max-cost <USD>`	Safety limit on total LLM spend
`--max-steps <N>`	Safety limit on total iterations
`--log-llm`	Log all LLM requests to DuckDB for post-analysis
`--single-file`	Force single-file mode (no DAG planning)
`--output-plan <FILE>`	Export the task plan as JSON before execution

Deferred Tests

By default, the agent runs tests (V_log) during each node's verification. With `--defer-tests`, V_log is set to 0.0 during node coding and tests only run at the final sheaf validation stage. This speeds up iteration:

```
perspt agent --yes --defer-tests -w /tmp/fast \
  "Build a CLI tool in Rust that converts CSV to JSON"
```

Trade-off

Deferred tests mean individual nodes may converge with untested code. The sheaf validation stage catches integration issues, but per-node test failures are discovered later.

Reading Structured Progress

In headless mode, Perspt emits structured progress to stderr:

```
[detect] Workspace: greenfield
[detect] Plugin: python (LSP: ty, tests: pytest, init: uv init --lib)
[plan] TaskPlan: 5 nodes, 1 plugin
[node-1] State: Generating -> Verifying -> Stable (V=0.00)
```

(continues on next page)

(continued from previous page)

```
[node-2] State: Generating -> Verifying -> Stable (V=0.00)
[node-3] State: Generating -> Verifying -> Retry (V=2.50)
[node-3] State: Generating -> Verifying -> Stable (V=0.00)
[sheaf] 3 validators, 0 failures, V_sheaf=0.00
[done] 5/5 nodes completed, session abc123
```

Checking Session Status

After a headless run, inspect the results:

```
# Session status
perspt status

# Recent ledger entries
perspt ledger --recent

# LLM usage statistics (if --log-llm was used)
perspt logs --stats

# Resume a failed session
perspt resume --last
```

Scripting Multiple Tasks

Run multiple agent tasks from a shell script:

```
#!/bin/bash
set -e
export GEMINI_API_KEY="your-key"

tasks=(
  "Create a Python CSV parser library"
  "Create a Rust JSON validator CLI"
  "Create a Python REST API with FastAPI"
)

for i in "${!tasks[@]}"; do
  dir="/tmp/project-$i"
  mkdir -p "$dir"
  perspt agent --yes --max-cost 2.0 -w "$dir" "${tasks[$i]}"
  echo "=== Project $i complete ==="
done
```

Safety Recommendations

1. **Always set cost limits** — `--max-cost 5.0` prevents runaway spending
2. **Use disposable directories** — Point `-w` to a fresh directory
3. **Review after generation** — Inspect the output before using it in production
4. **Use –log-llm** — Enables post-run analysis of what the agent did
5. **Set –max-steps** — Bounds the total number of retries across all nodes

See Also

- *Agent Mode Tutorial* — Interactive agent mode tutorial
- *SRBN Architecture* — SRBN technical details
- *Agent Options Reference* — Full agent CLI reference

5.1.4 Local Models with Ollama

Run Perspt with local models for privacy and offline usage.

Prerequisites

- Ollama installed
- Sufficient RAM for your chosen model (7B models: 8GB+, 70B models: 64GB+)

Setup

```
# Install Ollama (macOS)
brew install ollama

# Start the Ollama service
ollama serve

# Pull a model
ollama pull llama3.2
ollama pull codellama  # For coding tasks
```

Using Ollama with Perspt

Perspt auto-detects Ollama if no cloud API keys are set:

```
# Unset any cloud keys
unset OPENAI_API_KEY ANTHROPIC_API_KEY GEMINI_API_KEY

# Launch Perspt - auto-detects Ollama
perspt

# Or specify a model explicitly
perspt chat --model llama3.2
```

Agent Mode with Local Models

```
perspt agent --model codellama -w ./my-project "Create a Python utility"
```

Performance Note

Local models are slower and less capable than cloud models for complex agent tasks. For best results with agent mode, use a capable model (70B+ parameters) or use cloud models for the Architect and Verifier tiers:

```
export GEMINI_API_KEY="your-key"
perspt agent \
  --architect-model gemini-pro-latest \
  --actuator-model codellama \
  -w ./project "Create a utility"
```

Available Models

Popular Ollama models for use with Perspt:

Model	Size	Best For
`llama3.2`	3B/70B	General chat
`codellama`	7B/34B	Code generation
`deepseek-coder`	6.7B/33B	Code generation
`mistral`	7B	General purpose
`phi3`	3.8B	Lightweight tasks

5.1.5 Custom Workflows

Integrate Perspt into scripts, CI/CD pipelines, and automation.

Simple CLI for Scripting

The `simple-chat` command provides a Unix-friendly interface:

```
# Interactive
perspt simple-chat

# With session logging
perspt simple-chat --log-file session.txt
```

Batch Agent Runs

Run multiple agent tasks from a script:

```
#!/bin/bash
set -e
export GEMINI_API_KEY="your-key"

perspt agent --yes --max-cost 2.0 -w /tmp/proj1 "Create a Python CSV parser"
perspt agent --yes --max-cost 2.0 -w /tmp/proj2 "Create a Rust CLI calculator"
```

CI/CD Integration

Use headless mode in CI/CD pipelines:

```
# GitHub Actions example
name: Generate Boilerplate
on:
  workflow_dispatch:
    inputs:
      task:
        description: 'Task description'
        required: true

jobs:
```

(continues on next page)

(continued from previous page)

```
  generate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Install Perspt
        run: cargo install perspt
      - name: Run Agent
        env:
          GEMINI_API_KEY: ${{ secrets.GEMINI_API_KEY }}
        run: |
          perspt agent --yes --max-cost 5.0 \
            -w ./generated "${{ inputs.task }}"
      - name: Commit Results
        run: |
          git add generated/
          git commit -m "Generated: ${{ inputs.task }}"
```

Post-Run Analysis

After an agent run, use the management commands:

```
# Session status
perspt status

# LLM logs (requires --log-llm during the run)
perspt logs --tui
perspt logs --stats

# Ledger history
perspt ledger --recent
perspt ledger --stats

# Resume incomplete sessions
perspt resume --last
```

5.1.6 Example: Python ETL Pipeline

This tutorial demonstrates building a complete Python ETL (Extract, Transform, Load) pipeline using Perspt's agent mode.

Task Description

We will ask the agent to create a data pipeline that:

- Reads CSV files with validation
- Transforms data using Pydantic models
- Handles missing values and edge cases
- Includes comprehensive pytest tests

Running the Agent

```
export GEMINI_API_KEY="your-key"

perspt agent --yes -w /tmp/etl-demo \
  --architect-model gemini-pro-latest \
  --actuator-model gemini-3.1-flash-lite-preview \
  "Create a Python ETL pipeline package. It should:
   1. Read CSV files into Pydantic models for validation
   2. Transform records (clean nulls, normalize strings, compute derived fields)
   3. Write validated records to a new CSV
   4. Include a CLI entry point in src/main.py
   5. Use uv for dependency management with pandas and pydantic
   6. Include comprehensive pytest tests for each module"
```

Expected Output

The agent produces a project with `src/` layout:

```
/tmp/etl-demo/
+-- pyproject.toml          # [build-system] with uv_build backend
+-- uv.lock
+-- src/
|   +-- etl_pipeline/
|   |   +-- __init__.py
|   |   +-- core.py         # Main pipeline logic
|   |   +-- validator.py    # Pydantic models
|   |   +-- transformer.py  # Data transformations
|   +-- main.py             # CLI entry point
+-- tests/
    +-- test_core.py
    +-- test_validator.py
    +-- test_transformer.py
```

Verification

The SRBN engine verifies each node:

```
[node-1] Initialize Python package (V_boot=0.00)
[node-2] Create validator module (V_syn=0.00, V_log=0.00, V=0.00)
[node-3] Create transformer module (V_syn=0.00, V_log=0.00, V=0.00)
[node-4] Create core pipeline (V_syn=0.00, V_log=0.00, V=0.00)
[node-5] Create CLI entry point (V_syn=0.00, V=0.00)
[node-6] Write pytest tests (V_log=0.00, V=0.00)
[node-7] Integration wiring (V_sheaf=0.00)
```

After completion, verify manually:

```
cd /tmp/etl-demo
uv run pytest -v
# Expected: 15-20 tests passing
```

Key Observations

- The agent uses `uv init --lib` to create proper `src/` layout with `[build-system]` in `pyproject.toml`
- Ownership closure ensures no two nodes write to the same file
- The `ty` LSP server provides real-time type checking (V_syn)
- pytest provides test verification (V_log)
- All 7 SRBN nodes converge at V(x) = 0.00

See Also

- *Agent Mode Tutorial* — Agent mode tutorial
- *Example: Rust CLI Tool* — Rust project example

5.1.7 Example: Rust CLI Tool

Build a Rust command-line tool using Perspt's agent mode.

Task Description

We will ask the agent to create a CLI tool that converts CSV files to JSON.

Running the Agent

```
export GEMINI_API_KEY="your-key"

perspt agent --yes -w /tmp/csv2json \
  --architect-model gemini-pro-latest \
  --actuator-model gemini-3.1-flash-lite-preview \
  "Create a Rust CLI tool called csv2json that:
   1. Reads a CSV file from a path argument
   2. Converts each row to a JSON object
   3. Outputs JSON array to stdout or a file (--output flag)
   4. Handles errors gracefully with anyhow
   5. Uses clap for argument parsing
   6. Includes unit tests and integration tests"
```

Expected Output

```
/tmp/csv2json/
+-- Cargo.toml
+-- src/
|   +-- main.rs        # Entry point, clap args
|   +-- lib.rs          # Core conversion logic
|   +-- converter.rs    # CSV -> JSON conversion
+-- tests/
    +-- integration.rs  # Integration tests
```

Verification

The agent uses the `rust` plugin:

- **LSP**: `rust-analyzer` for V_syn
- **Tests**: `cargo test` for V_log
- **Init**: `cargo init` for V_boot

```
cd /tmp/csv2json
cargo test
cargo run -- --help
```

Key Observations

- The `rust` plugin selects `rust-analyzer` as the LSP server
- `cargo init` bootstraps the project structure (V_boot)
- `cargo test` runs both unit and integration tests (V_log)
- Ownership closure assigns each source file to exactly one node

See Also

- *Agent Mode Tutorial* — Agent mode tutorial
- *Example: Python ETL Pipeline* — Python project example

5.1.8 Example: Scientific Computing

This tutorial demonstrates using Perspt's headless agent mode for scientific computing tasks: CFD simulation setup and wind tunnel data analysis.

Experimental

These examples demonstrate advanced prompt patterns for scientific computing. Results depend on model capability and may require iteration. Treat them as starting points rather than guaranteed production workflows.

CFD Simulation Setup (Python)

Create a computational fluid dynamics simulation scaffolding:

```
export GEMINI_API_KEY="your-key"

perspt agent --yes --defer-tests -w /tmp/cfd-sim \
  --architect-model gemini-pro-latest \
  --actuator-model gemini-3.1-flash-lite-preview \
  "Create a Python CFD simulation package that:
   1. Defines a 2D grid mesh using numpy arrays
   2. Implements a simple Navier-Stokes solver (lid-driven cavity)
   3. Uses finite difference method for spatial discretization
   4. Includes a Jacobi iterative pressure solver
   5. Outputs velocity and pressure fields as numpy arrays
   6. Includes visualization with matplotlib (contour plots)
   7. Has pytest tests validating conservation laws
   8. Uses Pydantic for simulation parameters (Re, dt, grid_size)"
```

Expected output structure:

```
/tmp/cfd-sim/
+-- pyproject.toml
+-- src/
|   +-- cfd_sim/
|   |   +-- __init__.py
```

(continues on next page)

(continued from previous page)

```
|   |   +-- mesh.py          # Grid generation
|   |   +-- solver.py        # Navier-Stokes solver
|   |   +-- pressure.py      # Jacobi pressure solver
|   |   +-- visualization.py # matplotlib plotting
|   |   +-- parameters.py    # Pydantic config
|   +-- main.py
+-- tests/
    +-- test_mesh.py
    +-- test_solver.py
```

Wind Tunnel Data Analysis (Python)

Analyze experimental wind tunnel data:

```
perspt agent --yes --defer-tests -w /tmp/wind-tunnel \
  --architect-model gemini-pro-latest \
  --actuator-model gemini-3.1-flash-lite-preview \
  "Create a Python wind tunnel data analysis package that:
   1. Reads pressure tap data from CSV files
   2. Computes lift and drag coefficients (Cl, Cd) from pressure distributions
   3. Implements trapezoidal integration for force computation
   4. Generates polar plots of pressure coefficient (Cp) distribution
   5. Supports multiple angle-of-attack sweeps
   6. Includes Pydantic models for AirfoilData and TestConditions
   7. Exports results as JSON and matplotlib figures
   8. Has pytest tests with known NACA 0012 reference data"
```

Finite Element Solver (Rust)

Build a simple FEM solver in Rust:

```
perspt agent --yes -w /tmp/fem-solver \
  --architect-model gemini-pro-latest \
  --actuator-model gemini-3.1-flash-lite-preview \
  "Create a Rust finite element package that:
   1. Defines 2D triangular mesh elements
   2. Assembles global stiffness matrix for heat conduction
   3. Applies Dirichlet boundary conditions
   4. Solves using Gauss elimination (nalgebra)
   5. Outputs nodal temperatures
   6. Includes tests with known analytical solutions"
```

Tips for Scientific Tasks

1. **Be specific about algorithms** — Name the numerical method, discretization scheme, and solver type in the prompt
2. **Include validation criteria** — Reference known analytical solutions or benchmark data for tests
3. **Use –defer-tests** — Scientific tests often depend on numerical tolerances that need tuning after initial generation
4. **Set higher cost limits** — Scientific code often requires more iterations for convergence: `--max-cost 10.0`
5. **Review carefully** — Numerical correctness requires domain expertise beyond what the LSP can verify

See Also

- *Headless Mode* — Headless mode guide
- *Agent Mode Tutorial* — Interactive agent mode

5.1.9 Monitoring Agent Execution with the Dashboard

This tutorial walks through using the Perspt web dashboard to observe an agent session in real time.

Prerequisites

- Perspt installed (`cargo install --path crates/perspt-cli`)
- A project directory to work in

Step 1: Start an Agent Session

```
perspt agent -w ./myproject "Create a REST API in Rust"
```

The agent will begin planning and executing tasks. Leave this terminal running.

Step 2: Launch the Dashboard

Open a **new terminal** and run:

```
perspt dashboard
```

You should see:

```
Perspt dashboard listening on http://127.0.0.1:3000
```

Step 3: Open the Overview

Navigate to `http://localhost:3000` in your browser. The Overview page shows your active session with:

- **Status badge** — "running" (blue) while the agent works
- **Node count** — completed/total with failed count if any
- **Budget** — steps used and cost consumed

Step 4: Explore the DAG

Click **DAG** next to your session. Node cards show the task decomposition:

- Green border — committed/verified nodes
- Blue border — currently running
- Red border — failed (will be retried)

The edge table below shows parent → child relationships.

Step 5: Watch Energy Convergence

Click **Energy** to see per-node energy components. The agent aims to minimize V_total across all nodes. Watch values decrease as the verifier-guided correction loop runs.

Step 6: Review LLM Telemetry

The **LLM** page shows:

- Total requests, tokens in/out, and cumulative latency in the stats bar
- Individual request details with model, prompt preview, and response preview

This is useful for auditing LLM usage and identifying expensive calls.

Step 7: Check Sandbox Branches

The **Sandbox** page lists provisional branches the agent is exploring. Active branches have sandbox directories; merged/flushed branches show their final state.

Step 8: Understand Decisions

The **Decisions** page reveals the agent's internal reasoning:

- **Escalation Reports** — when nodes were escalated for re-planning
- **Sheaf Validations** — multi-source consistency checks
- **Rewrites** — DAG modifications (requeued/inserted nodes)
- **Plan Revisions** — architect plan amendments with reasons
- **Repair Footprints** — correction attempts and diagnoses
- **Verification Results** — syntax, build, test, and lint results

Step 9: Monitor Live Updates

The dashboard receives Server-Sent Events (SSE) from the server every 2 seconds. You can keep the browser open and watch as the agent progresses through its task.

When the agent completes, the session status changes to "completed" (green badge) on the Overview page.

Chapter 6

User Guide

In-depth coverage of every Perspt feature.

Basic Usage TUI navigation, chat commands, and session management.

Basic Usage Agent Mode Experimental SRBN lifecycle, DAG planning, node verification, and review.

Agent Mode Advanced Features Per-tier models, energy tuning, ledger, resume, and cost control.

Advanced Features Providers Configure OpenAI, Anthropic, Gemini, Ollama, and more.

Providers Troubleshooting Diagnose and fix common issues.

Troubleshooting Dashboard Real-time web monitoring of agent sessions.

Dashboard

6.1 Basic Usage

Perspt offers two interactive modes: the **TUI** (rich terminal UI) and **simple-chat** (plain-text streaming).

6.1.1 Launching the TUI

```
# Auto-detect provider from env keys
perspt

# Explicit provider + model
perspt chat --provider-type anthropic --model claude-sonnet-4-20250514

# Override API key
perspt chat --api-key sk-xxx --model gpt-4.1
```

The TUI provides:

- Markdown rendering (code blocks, headers, lists, bold, italic)
- Real-time response streaming
- Scroll navigation
- Status bar (provider, model, streaming indicator)

6.1.2 Keyboard Shortcuts

Key	Action
Enter	Send message
Ctrl+J	Insert newline in input
Page Up / Down	Scroll chat history
Ctrl+Up / Down	Scroll by one line
Home / End	Jump to top / bottom
Esc or **Ctrl+Q**	Quit
Ctrl+C	Cancel current stream

6.1.3 Chat Commands

Command	Description
`/save`	Save conversation to a timestamped text file
`exit` or `quit`	Exit the application

6.1.4 Simple CLI Mode

For a minimal text interface suitable for piping and logging:

```
perspt simple-chat
perspt simple-chat --log-file session.txt
```

Type your message, press Enter. Responses stream to stdout. Type `exit` or press `Ctrl+D` to quit.

6.1.5 Provider Auto-Detection

Perspt checks environment variables in this priority order:

Env Variable	Provider	Default Model
`ANTHROPIC_API_KEY`	Anthropic	`claude-sonnet-4-20250514`
`OPENAI_API_KEY`	OpenAI	`gpt-4.1`
`GEMINI_API_KEY`	Gemini	`gemini-3.1-flash-lite-preview`
`GROQ_API_KEY`	Groq	`llama-3.3-70b-versatile`
`COHERE_API_KEY`	Cohere	`command-r-plus`
`XAI_API_KEY`	xAI	`grok-3-mini-fast`
`DEEPSEEK_API_KEY`	DeepSeek	`deepseek-chat`
(none)	Ollama (local)	`llama3.2`

See *Providers* for full provider details.

6.2 Agent Mode

The agent command activates the experimental SRBN (Stabilized Recursive Barrier Network) engine for autonomous multi-file code generation.

6.2.1 Launching Agent Mode

```
perspt agent -w <DIR> "<task>"

# Examples
perspt agent -w ./my-project "Create a Python REST API"
perspt agent -y -w /tmp/demo "Build a Rust CLI tool"
```

6.2.2 Core Workflow

The SRBN agent follows the PSP-5 lifecycle:

1. **Detection** — Identify workspace state (greenfield/brownfield) and select language plugins
2. **Planning** — Based on the auto-selected `PlanningPolicy`, either:
   - **LocalEdit** — Skip Architect, create a single-node graph
   - **FeatureIncrement / LargeFeature / GreenfieldBuild / ArchitecturalRevision** — Architect decomposes the task into a DAG of nodes with assigned classes.

   A `FeatureCharter` is created with policy-derived limits (max modules, files, and revisions) to constrain plan scope.
3. **Execution** — For each node in topological order:
   a. Actuator generates a multi-artifact bundle (writes, diffs, commands)
   b. Bundle is applied transactionally
   c. Verification computes Lyapunov energy: $V(x) = \alpha \cdot V_{syn} + \beta \cdot V_{str} + \gamma \cdot V_{log} + V_{boot} + V_{sheaf}$
   d. If $V(x) < \varepsilon$ (default 0.10), node is stable; otherwise retry
4. **Sheaf Validation** — Cross-node contract verification
5. **Review** — In interactive mode: grouped-diff modal with approve/reject/correct
6. **Commit** — Stable nodes are committed to the Merkle ledger

6.2.3 Node Classes

Each DAG node belongs to a class that governs scheduling and verification:

Class	Description
Interface	Public API definitions, type signatures, traits. Scheduled first.
Implementation	Internal logic. May depend on Interface nodes.
Integration	Wiring, main entry, config assembly. Scheduled last.

6.2.4 Artifact Bundle Protocol

Each node produces a bundle with three artifact types:

Type	Description
`write`	Create a new file with full content
`diff`	Modify an existing file (unified diff format)
`command`	Execute a shell command (e.g., `uv add pandas`)

Ownership closure ensures no two nodes own the same file.

6.2.5 Review Modal (Interactive)

When running without `--yes`, the review modal presents:

```
Review Node 3: Implement data transformer
──────────────────────────────────────────

Bundle: 1 created, 1 modified
+ src/transformer.py    [create] (45 lines)
~ src/pipeline.py       [diff]   (+3, -1)

Verification: V_syn OK | V_str OK | V_log OK | V_boot OK
Energy: V(x) = 0.00

[y] Approve  [n] Reject  [c] Correct  [e] Edit  [d] Diff
```

- **y** — Approve and commit to ledger
- **n** — Reject and regenerate
- **c** — Send feedback for correction
- **e** — Open files in your editor
- **d** — Toggle full diff view

6.2.6 Session Management

```
# Check session state (per-node counts, energy, escalations, correction attempts)
perspt status

# Abort the current session
perspt abort

# Resume with trust context (shows escalation count, energy, retries)
perspt resume --last
```

When resuming, the `BudgetEnvelope` (step, cost, and revision caps) is restored from the database so limits continue from where the session left off.

6.2.7 Dashboard Monitoring

While an agent session runs, you can observe it in a browser:

```
# In a separate terminal
perspt dashboard
```

Open `http://localhost:3000` to see the Overview, DAG, Energy, LLM, Sandbox, and Decisions pages. The dashboard reads the session store in read-only mode so it never interferes with the running agent.

See *Dashboard* for full details.

6.2.8 Speculator Lookahead

For complex policies (`LargeFeature`, `GreenfieldBuild`, `ArchitecturalRevision`), the Speculator tier runs a fast lookahead before each node's Actuator generation. It examines pending child nodes and produces risk hints that are injected into the Actuator's prompt, helping avoid downstream breakage. Simpler policies (`LocalEdit`, `FeatureIncrement`) skip the speculator to reduce latency and cost.

See *Advanced Features* for model tiers, energy tuning, and cost controls.

6.2.9 Correction Observability (PSP-7)

PSP-7 adds structured telemetry to the correction loop. Every correction attempt is persisted with the parse result state, retry classification, and energy snapshot.

The `perspt status` command now shows:

- **Step Timeline** — per-step-type counts and total execution time
- **Correction Attempts** — per-node accepted/rejected attempt counts

The dashboard **Decisions** page includes a dedicated Correction Attempts table with node, attempt number, parse state, and rejection reason.

In headless mode, the agent summary emits `[STEPS]` and `[CORRECTIONS]` lines for CI integration:

```
[STEPS] 15 records, 42.3s total
[CORRECTIONS] 2 node(s) needed correction, 5 total attempts
```

6.2.10 Live Dashboard Monitoring

Use `--dashboard` to start the web monitoring dashboard alongside the agent:

```
perspt agent --dashboard "Build a REST server"
# Open http://127.0.0.1:3000 in a browser
```

The embedded dashboard opens a read-only DuckDB connection to the same database the agent writes to, providing real-time views of DAG topology, energy convergence, LLM telemetry, and correction-attempt history. Use `--dashboard-port` to change the port. The server stops when the agent exits.

6.3 Advanced Features

6.3.1 Per-Tier Model Selection

Assign different models to each SRBN tier:

```
perspt agent \
  --architect-model gemini-pro-latest \
  --actuator-model gemini-3.1-flash-lite-preview \
  --verifier-model gemini-pro-latest \
  --speculator-model gemini-3.1-flash-lite-preview \
  -w ./project "Task description"
```

Each tier also supports a `--<tier>-fallback-model` flag for automatic failover:

```
perspt agent \
  --architect-model gemini-pro-latest \
  --architect-fallback-model gemini-3.1-flash-lite-preview \
  -w ./project "Task"
```

6.3.2 Energy Weights

Customize the Lyapunov energy function:

$$V(x) = \alpha \cdot V_{\text{syn}} + \beta \cdot V_{\text{str}} + \gamma \cdot V_{\text{log}} + V_{\text{boot}} + V_{\text{sheaf}}$$

```
# Default: alpha=1.0, beta=0.5, gamma=2.0
perspt agent --energy-weights "1.0,0.5,2.0" -w . "Task"

# Prioritize tests (higher gamma)
perspt agent --energy-weights "0.5,0.5,3.0" -w . "Add tests"

# Prioritize type safety (higher alpha)
perspt agent --energy-weights "2.0,1.0,0.5" -w . "Add types"
```

6.3.3 Stability Threshold

```
# Default: epsilon = 0.10
perspt agent --stability-threshold 0.10 -w . "Precise task"

# Relaxed for prototyping
perspt agent --stability-threshold 0.5 -w . "Quick prototype"
```

6.3.4 Cost and Step Limits

```
# Cap total LLM spend at $5
perspt agent --max-cost 5.0 -w . "Large refactor"

# Cap total iterations across all nodes
perspt agent --max-steps 20 -w . "Iterative task"
```

6.3.5 Complexity Control

```
# Set complexity threshold
perspt agent -k 3 -w . "Simple task"

# Explicit complexity estimation
perspt agent --complexity medium -w . "Medium task"
```

6.3.6 Deferred Testing

Skip unit tests during per-node verification; only run them at sheaf validation:

```
perspt agent --defer-tests -w . "Speed-optimized generation"
```

This sets V_log = 0.0 during node coding. Tests run only at the final sheaf stage.

6.3.7 Verifier Strictness

```
# Default strictness
perspt agent --verifier-strictness default -w . "Task"

# Strict: fail on any warning
perspt agent --verifier-strictness strict -w . "Production code"

# Minimal: only fail on errors
perspt agent --verifier-strictness minimal -w . "Prototype"
```

6.3.8 LLM Logging and Analytics

```
# Enable LLM call logging to DuckDB
perspt agent --log-llm -w . "Task"

# Interactive log browser
perspt logs --tui

# Show most recent session
perspt logs --last

# Usage statistics (tokens, cost, timing)
perspt logs --stats
```

6.3.9 Merkle Ledger

Every stable node is committed to a content-addressed Merkle ledger stored in DuckDB. This provides:

- **Auditability** — Full trace of what each node produced
- **Rollback** — Restore to any point in the session
- **Resume** — Continue interrupted sessions with verified context

```
perspt ledger --recent
perspt ledger --stats
```

6.3.10 Single-File Mode

Force the agent to produce a single file without DAG planning:

```
perspt agent --single-file -w . "Create a Python utility script"
```

6.3.11 Plan Export

Save the task plan as JSON before execution:

```
perspt agent --output-plan plan.json -w . "Create a web app"
cat plan.json
```

6.3.12 Execution Modes

Mode	Behavior
`cautious`	Prompt for approval on every node
`balanced`	Prompt when complexity exceeds threshold K (default)
`yolo`	Auto-approve everything without review

6.3.13 Planning Policy

The orchestrator automatically selects a `PlanningPolicy` based on workspace state. The policy controls which agent tiers are activated:

Policy	Architect	Speculator	When Selected
LocalEdit	Skipped	Skipped	Small, localized changes (single-node graph)
FeatureIncrement	Active	Skipped	Existing projects (default)
LargeFeature	Active	Active	Complex multi-module tasks
GreenfieldBuild	Active	Active	New project (no existing files)
ArchitecturalRevision	Active	Active	Cross-cutting redesign

When the Speculator is active, it runs a fast lookahead before each node's code generation, producing risk hints about downstream impacts. This adds latency but improves first-pass correctness for complex DAGs.

A **FeatureCharter** is auto-created with policy-derived limits before planning:

- **LocalEdit**: max 1 module, 5 files, 3 revisions
- **FeatureIncrement**: max 10 modules, 30 files, 5 revisions
- **LargeFeature / GreenfieldBuild / ArchitecturalRevision**: max 25 modules, 80 files, 10 revisions

6.4 Providers

Perspt supports multiple LLM providers through the `genai` crate.

6.4.1 Supported Providers

Provider	Env Variable	Default Model	Notes
OpenAI	`OPENAI_API_KEY`	`gpt-4.1`	GPT-4, GPT-4o, o-series
Anthropic	`ANTHROPIC_API_KEY`	`claude-son-net-4-20250514`	Claude 4 family
Google Gemini	`GEMINI_API_KEY`	`gemini-3.1-flash-lite-preview`	Gemini 2.x family
Groq	`GROQ_API_KEY`	`llama-3.3-70b-versatile`	Llama, Mixtral
Cohere	`COHERE_API_KEY`	`command-r-plus`	Command R family
xAI	`XAI_API_KEY`	`grok-3-mini-fast`	Grok models
DeepSeek	`DEEPSEEK_API_KEY`	`deepseek-chat`	DeepSeek models
Ollama	*(none)*	`llama3.2`	Local models via Ollama

6.4.2 Configuration Methods

1. Environment Variables (recommended):

```
export GEMINI_API_KEY="your-key"
perspt
```

2. CLI Flags:

```
perspt chat --api-key "your-key" --provider-type openai --model gpt-4.1
```

3. Config File (`~/.config/perspt/config.json`):

```
{
  "default_provider": "anthropic",
```

(continues on next page)

(continued from previous page)

```
  "default_model": "claude-sonnet-4-20250514",
  "api_key": "sk-ant-xxx"
}
```

Priority order: CLI flags > environment variables > config file > auto-detection.

6.4.3 Listing Available Models

```
perspt --list-models
```

6.4.4 Provider-Specific Notes

OpenAI

```
export OPENAI_API_KEY="sk-xxx"
perspt chat --model gpt-4.1
```

Anthropic

```
export ANTHROPIC_API_KEY="sk-ant-xxx"
perspt chat --model claude-sonnet-4-20250514
```

Google Gemini

```
export GEMINI_API_KEY="AIza..."
perspt chat --model gemini-3.1-flash-lite-preview
```

Ollama (Local)

```
ollama serve
ollama pull llama3.2
perspt chat --model llama3.2
```

No API key required. Perspt auto-detects Ollama as the fallback provider.

6.5 Troubleshooting

6.5.1 API Key Issues

Symptom: "No API key found" error.

Solutions:

1. Check env var is exported: `echo $GEMINI_API_KEY`
2. Verify spelling: `ANTHROPIC_API_KEY` (not `ANTHROPIC_KEY`)
3. Pass explicitly: `perspt chat --api-key "your-key"`
4. Check config file: `~/.config/perspt/config.json`

6.5.2 Connection Errors

Symptom: "Connection refused" or timeouts.

Solutions:

1. Check internet connectivity
2. For Ollama: ensure `ollama serve` is running
3. Check firewall/proxy settings
4. Try a different provider

6.5.3 Agent Mode Issues

Agent stuck in retry loop:

1. Check tool prerequisites: `which uv`, `which cargo`, `which node`
2. Check LSP is functioning: `ty check .` or `cargo check`
3. Lower stability threshold: `--stability-threshold 0.5`
4. Use `--defer-tests` to skip V_log during coding
5. Check `perspt status` for escalation details

High energy despite clean code:

1. Run tests manually: `uv run pytest -v` or `cargo test`
2. Check for LSP diagnostics: `ty check .`
3. Adjust energy weights: `--energy-weights "0.5,0.5,1.0"`
4. Verify contract compliance

Plugin not detected:

1. Ensure required binaries are installed in PATH
2. Check workspace has expected marker files (`Cargo.toml`, `pyproject.toml`)
3. Run `perspt status` to see active plugins

6.5.4 TUI Rendering Issues

Symptom: Garbled output, incorrect colors.

Solutions:

1. Ensure terminal supports 256 colors: `echo $TERM`
2. Try a different terminal emulator
3. Fallback to simple CLI: `perspt simple-chat`
4. Check for conflicting terminal multiplexer settings

6.5.5 Degraded Verification

When tool binaries (`ty`, `cargo`, `pytest`) are missing, the agent enters **degraded verification mode**:

- V_syn is estimated via heuristic pattern matching (regex-based)
- V_log uses `exit 0` stubs
- V_boot skips missing commands

This allows the agent to function, but with lower verification confidence.

To restore full verification, install the required tools:

```
# Python projects
pip install ty pytest

# Rust projects
rustup component add rust-analyzer

# Node.js projects
npm install -g typescript
```

6.5.6 Session Recovery

If a session is interrupted:

```
# Check what's in progress
perspt status

# Resume the last session (shows trust context first)
perspt resume --last

# Or abort and start fresh
perspt abort
```

6.5.7 Getting Help

```
perspt --help
perspt agent --help
perspt chat --help
```

For more details, see:

- *CLI Reference* — Full CLI reference
- *Advanced Troubleshooting* — Advanced troubleshooting

6.6 Dashboard

The Perspt web dashboard provides real-time browser-based monitoring of agent execution. Launch it alongside a running agent session to observe DAG topology, energy convergence, LLM telemetry, sandbox branches, and decision traces.

6.6.1 Launching the Dashboard

In a separate terminal from your agent session:

```
perspt dashboard
```

This starts an Axum web server on `http://127.0.0.1:3000`. Open that URL in your browser. The dashboard reads the DuckDB session store in read-only mode, so it can run safely alongside the agent.

To use a different port:

```
perspt dashboard --port 8080
```

6.6.2 Dashboard Pages

Overview — Lists all recent sessions with status badges (running, completed, failed), node completion counts, and budget consumption. Click any session to drill into its sub-pages.

DAG Topology — Shows node cards colored by state (green for committed/verified, red for failed, blue for running). Displays the task graph edges below. Useful for understanding the agent's decomposition of work.

Energy Convergence — Displays per-node energy components: V_syn (syntax), V_str (structure), V_log (tests), V_boot (bootstrap), and V_sheaf (sheaf validation). The V_total column shows the combined energy value.

LLM Telemetry — Summary stats bar showing total requests, tokens in/out, and cumulative latency. Below, a table of individual LLM requests with model, node, token counts, latency, and prompt/response previews.

Sandbox Monitoring — Lists provisional branches with their state (active, merged, flushed) and sandbox directories. Useful for tracking which code changes are being explored.

Decision Trace — Collapsible sections for each decision category: escalation reports, sheaf validations, DAG rewrites, plan revisions, repair footprints, and verification results. Each section shows relevant details in tabular form.

6.6.3 Authentication

By default, the dashboard runs without authentication on localhost. This is safe for local development.

To require a password, add to your `config.toml`:

```
[dashboard]
password = "your-secret-password"
```

When a password is configured, visiting any page redirects to `/login`. After entering the correct password, a session cookie is set and subsequent requests pass through.

Cookie attributes:

- `HttpOnly` — not accessible via JavaScript
- `SameSite=Lax` — CSRF protection
- `Secure` — set when not on localhost
- `Path=/` — applies to all dashboard routes

6.6.4 Using with a Running Agent

The typical workflow:

1. Start an agent session:

   ```
   perspt agent -w ./myproject "Create a REST API server"
   ```

2. In another terminal, launch the dashboard:

   ```
   perspt dashboard
   ```

3. Open `http://localhost:3000` in your browser.
4. The dashboard updates via Server-Sent Events (SSE) every 2 seconds, showing live node state changes as the agent works.

6.6.5 Viewing Historical Sessions

The Overview page shows the 50 most recent sessions. Sessions from past agent runs remain in the DuckDB database and can be browsed even after the agent has stopped.

Chapter 7

Concepts

Understanding the foundations of Perspt.

SRBN Architecture The theoretical framework and its experimental implementation in Perspt.

SRBN Architecture Workspace Crates The 9-crate modular architecture.

Workspace Crates PSP Process Perspt Specification Proposals for design and development.

PSP Process

7.1 SRBN Architecture

The **Stabilized Recursive Barrier Network (SRBN)** is the theoretical framework behind Perspt's experimental autonomous coding agent. SRBN is based on the paper *"Stability is All You Need: Lyapunov-Guided Hierarchies for Long-Horizon LLM Reliability"* by **Vikrant R. and Ronak R.** (pre-publication), which reformulates LLM agency as a sheaf-theoretic control problem and proves Input-to-State Stability (ISS) under persistent noise. Perspt's implementation of this framework is defined by **PSP-5** (Perspt Specification Proposal 5).

Theory vs. Implementation

This page describes both the SRBN paper's theoretical model and how Perspt's PSP-5 runtime implements it. Where a claim comes from the paper's formal proofs, it is noted as a **paper result**. Where PSP-5 makes engineering choices that approximate or extend the theory, those are noted as **implementation details**. The theoretical framework is mature; empirical benchmarks on Perspt's implementation have not yet been published.

7.1.1 Overview

The SRBN paper models coding tasks as a directed acyclic graph (DAG) of nodes with a sheaf structure that enforces consistency across shared boundaries. PSP-5 implements this model concretely: each node owns a set of output files (ownership closure), generates a multi-artifact bundle, and must pass multi-stage verification before its energy falls below the convergence threshold. Only then is the node committed to the Merkle ledger.

7.1.2 The Control Loop (PSP-5 Implementation)

The SRBN control loop as implemented by PSP-5 executes seven phases for each task:

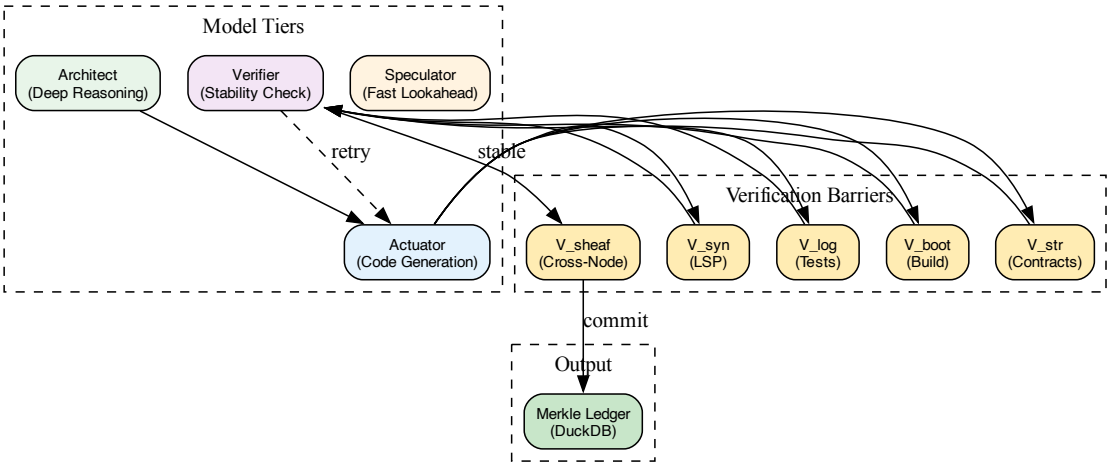


Figure 1: SRBN Architecture

#	Phase	Description
1	**Detection**	Inspect the repository. Select language plugins (Rust, Python, JS, Go) based on existing files or the task description. Each plugin provides an LSP server, test runner, and init command.
2	**Planning**	Architect model decomposes the task into a `TaskPlan` DAG. Each node has an ID, goal, context files, output files, dependencies, task type, and node class (Interface, Implementation, or Integration). The **ownership closure** rule ensures each output file appears in exactly one node. Planning is gated by `PlanningPolicy`: `LocalEdit` skips the Architect and creates a single-node graph; all other policies run the full Architect decomposition. A **FeatureCharter** is auto-created with policy-derived limits (`max_modules`, `max_files`, `max_revisions`) before planning begins.
3	**Generation**	Actuator model generates a multi-artifact bundle per node. The bundle is a JSON structure with `write`, `diff`, and `command` operations. All files are written transactionally.
4	**Verification**	Compute five energy components: V_syn (LSP diagnostics), V_str (contract violations), V_log (test failures), V_boot (init/build exit codes), and V_sheaf (cross-node consistency). Total energy is V(x).
5	**Convergence**	If V(x) > epsilon, generate a grounded correction prompt containing the specific error messages and retry. Bounded by `RetryPolicy` per error type.
6	**Sheaf Validation**	After all nodes converge individually, run cross-node consistency checks. Validates import paths, shared type signatures, and interface-seal digests.
7	**Commit & Outcome**	Record each node's terminal state in the Merkle ledger. Nodes that converge (V(x) ≤ ε) are committed as `Completed`; nodes whose retries are exhausted are recorded as `Escalated`. After all nodes are processed, the orchestrator derives a `SessionOutcome` from completed/escalated counts: `Success` (all completed), `PartialSuccess` (some escalated), or `Failed` (none completed). Emit `Complete` event with the derived outcome.

7.1.3 Lyapunov Energy

The stability of generated code is measured using a Lyapunov energy function, adapted from the paper's sheaf-theoretic formulation into five concrete verification barriers:

Energy Formula

$$V(x) = \alpha \cdot V_{syn} + \beta \cdot V_{str} + \gamma \cdot V_{log} + V_{boot} + V_{sheaf}$$

Default weights: alpha = 1.0, beta = 0.5, gamma = 2.0

Components

Component	Source	Description
V_syn	LSP Diagnostics	Count of errors and warnings from the language server (`rust-analyzer`, `ty`, `pyright`, `typescript-language-server`, `gopls`).
V_str	Contract Verification	Violations of `BehavioralContract` constraints: interface signatures, invariants, and forbidden patterns.
V_log	Test Failures	Weighted sum of test failures. Critical tests carry weight 10, high-priority 3, low-priority 1. Computed via pytest or `cargo test`.
V_boot	Bootstrap Commands	Non-zero exit codes from init commands (`uv init --lib`, `cargo init`), build commands, and dependency installs.
V_sheaf	Cross-Node Consistency	Failures from sheaf validators: import-path resolution, shared-type agreement, and interface-seal digest mismatches.

Convergence Criterion

The system is considered stable when:

$$V(x) \leq \varepsilon$$

Default: epsilon = 0.10. Configurable via `--stability-threshold`.

7.1.4 Node Classes

PSP-5 introduces three node classes that govern execution order and verification:

Class	Description
Interface	Define exported signatures, schemas, and seals. Must be committed before dependent Implementation nodes can proceed. Produces an interface-seal digest.
Implementation	Operate on owned files using sealed interfaces from parent nodes. The bulk of code generation happens here.
Integration	Reconcile cross-owner boundaries after all dependent nodes converge. Used for multi-language projects or cross-module wiring.

7.1.5 Ownership Closure

The **ownership closure** rule is a fundamental invariant of PSP-5:

> *Each output file appears in exactly one node's* `output_files` *list.*

This prevents conflicting writes. When the Architect generates a task plan, the orchestrator validates ownership closure before execution begins. If two nodes claim the same file, the plan is rejected and re-generated.

7.1.6 Model Tiers

SRBN uses four specialized model tiers. Each tier can be configured independently:

Tier	Purpose	Default Model
Architect	Deep reasoning, task decomposition, DAG planning	`gemini-pro-latest`
Actuator	Code generation, artifact bundle emission	`gemini-3.1-flash-lite-preview`
Verifier	LSP diagnostics, contract checking, energy computation	`gemini-pro-latest`
Speculator	Fast lookahead, provisional branch prediction	`gemini-3.1-flash-lite-preview`

Configure per-tier models via CLI:

```
perspt agent \
  --architect-model gemini-pro-latest \
  --actuator-model gemini-3.1-flash-lite-preview \
  --verifier-model gemini-pro-latest \
  --speculator-model gemini-3.1-flash-lite-preview \
  "Build a REST API"
```

Each tier also supports a fallback model (`--architect-fallback-model`, etc.).

7.1.7 Planning Policy

The `PlanningPolicy` enum adapts the agent phase stack based on task scale:

Policy	Architect	Speculator	Description
LocalEdit	No	No	Small, localized changes. Skips task decomposition; uses a single-node graph.
FeatureIncrement (default)	Yes	No	Mid-size features. Architect decomposes, Actuator implements, Verifier checks.
LargeFeature	Yes	Yes	Full SRBN loop including speculator lookahead for downstream risk hints.
GreenfieldBuild	Yes	Yes	New project. Full stack with workspace bootstrap node first.
ArchitecturalRevision	Yes	Yes	Cross-cutting redesign. Plan-first with speculator risk analysis.

7.1.8 PSP-7: Robust Correction Loop Contracts

PSP-7 extends the SRBN runtime with three hardening layers: a typed parse pipeline, a prompt compiler with provenance tracking, and structured correction telemetry.

Typed Parse Pipeline (Fail-Closed Parsing)

LLM responses are processed through a five-layer typed pipeline that replaces the legacy `extract_all_code_blocks_from_response()` fallback. Each layer returns a `ParseResultState` that classifies the outcome:

Layer	Description
A (Raw Capture)	Hash, length, and first-line fingerprint of the raw response.
B (Path Normalization)	Strip backticks, quotes, and markdown formatting from file paths.
C (Strict JSON)	Attempt to parse the response as a structured JSON artifact bundle.
D (Tolerant Recovery)	Recognize `### File:`, `File:`, and `Diff:` headings to extract structured content. Never invents filenames — unnamed blocks produce `None`.
E (Semantic Validation)	Plugin-driven ownership closure, `legal_support_files()` checks, `dependency_command_policy()` enforcement.

`ParseResultState` has six variants: `StructuredOk`, `TolerantRecoveryOk`, `NoStructuredPayload`, `SchemaInvalid`, `SemanticallyRejected`, `EmptyResponse`. Each variant carries a `RetryClassification` that guides the correction loop: `Retarget`, `SupportFiles`, `Replan`, or `FatalBudget`.

Active V_boot

PSP-7 separates bootstrap failures from code errors. After auto-repair re-verification, if the verifier profile is fully degraded or missing-crate/module failures persist, `V_boot` is set independently rather than being folded into `V_syn`. This gives the correction loop a dedicated signal for infrastructure problems.

Sheaf Pre-Check

After a node converges ($V(x) \leq \varepsilon$) but before the full sheaf validation pass, a fast structural pre-check verifies that output artifacts declare consistent imports and exports with the ownership manifest. If the pre-check fails, the node re-enters `step_converge()` with sheaf-specific evidence. A retry guard (max 1 sheaf pre-check retry) prevents infinite loops.

Prompt Compiler

PSP-7 replaces the template-constant approach with a typed prompt compiler:

```
compile(intent: PromptIntent, evidence: &PromptEvidence) -> CompiledPrompt
```

The compiler accepts 13 `PromptIntent` variants (architect, actuator, verifier, correction, speculator, solo, bundle retarget, project naming) and emits a `CompiledPrompt` with the assembled prompt text plus `PromptProvenance` metadata (template ID, evidence hashes, compiler version). Plugin `correction_prompt_fragment()` and `legal_support_files()` are injected into correction prompts automatically.

Correction Telemetry

Every correction attempt is recorded as a `CorrectionAttemptRow` in the DuckDB store:

- `parse_state` — which `ParseResultState` was returned
- `retry_classification` — how the failure was classified
- `response_fingerprint` — hash of the raw LLM response
- `response_length` — byte length for detecting degenerate responses
- `energy_json` — energy components snapshot after verification
- `accepted` / `rejection_reason` — whether the attempt was committed

Additionally, `srbn_step_records` track per-node execution steps (speculate, verify, converge, sheaf_validate, commit) with timing, energy snapshots, and attempt counts. These records are surfaced by `perspt status`, the dashboard decisions page, and the headless agent summary.

The policy is auto-selected based on workspace state (greenfield vs existing project). `needs_architect()` gates whether the Architect tier runs; `needs_speculator()` gates the speculator lookahead call.

Feature Charter

Before architect planning begins, the orchestrator creates a `FeatureCharter` with policy-derived defaults:

- **LocalEdit**: max 1 module, 5 files, 3 revisions
- **FeatureIncrement**: max 10 modules, 30 files, 5 revisions
- **LargeFeature / GreenfieldBuild / ArchitecturalRevision**: max 25 modules, 80 files, 10 revisions

The charter gates the plan: if the Architect produces a plan exceeding the charter's module or file budget, a warning is emitted. Language constraints are derived from active plugins.

7.1.9 Retry Policy

SRBN implements bounded retries per error type:

Error Type	Max Retries	Escalation
Compilation errors (LSP)	3	Escalate to user with diagnostic context
Tool failures (file ops)	5	Escalate with error logs
Review rejections (user)	3	Escalate with diff summary

When retries are exhausted, the node transitions to **Escalated** state. Escalated nodes do not block subsequent nodes. The orchestrator tracks completed and escalated counts and derives the final `SessionOutcome`: `Success` if all nodes completed, `PartialSuccess` if some escalated, or `Failed` if none completed. In headless mode (`--yes`), escalations are logged and the session exits with a non-zero code when the outcome is not `Success`.

7.1.10 Artifact Bundle Protocol

The Actuator emits a JSON artifact bundle for each node:

```
{
  "artifacts": [
    {
      "path": "src/lib.rs",
      "operation": "write",
      "content": "pub fn add(a: i32, b: i32) -> i32 { a + b }"
    },
    {
```

(continues on next page)

(continued from previous page)

```
      "path": "src/main.rs",
      "operation": "diff",
      "patch": "--- a/src/main.rs\n+++ b/src/main.rs\n@@ -1 +1,3 @@..."
    }
  ],
  "commands": [
    "cargo build"
  ]
}
```

Operations:

- **write** — Create or overwrite a file with the given content
- **diff** — Apply a unified diff patch to an existing file
- **command** — Execute a shell command (validated by policy engine)

All artifacts are applied transactionally. If any operation fails, the entire bundle is rolled back.

7.1.11 Plugin-Driven Verification

Language plugins determine the verification toolchain:

Plugin	LSP Server	Test Runner	Init Command	Required Binaries
Rust	`rust-analyzer`	`cargo test`	`cargo init`	`cargo`, `rustc`
Python	`ty` or `pyright`	`pytest`	`uv init --lib`	`uv`, `python3`
JavaScript	`typescript-language-server`	`npm test`	`npm init -y`	`node`, `npm`
Go	`gopls`	`go test`	`go mod init`	`go`

The plugin is selected automatically during the Detection phase based on existing project files or the task description. Multi-language projects activate multiple plugins simultaneously.

7.1.12 Degraded Verification

When a verification tool is unavailable (e.g., `ty` not installed), the SRBN engine falls back to degraded mode:

- **Sensor fallback**: If the primary LSP server is not found, try a secondary (e.g., `pyright` instead of `ty`). Emit a `SensorFallback` event.
- **Degraded stages**: If no LSP server is available at all, V_syn is set to 0.0 and the stage is marked degraded. Energy convergence proceeds without that component.
- **Stability blocked**: If too many stages degrade, the node cannot converge and is escalated.

7.1.13 Merkle Ledger

All changes are recorded in a DuckDB-backed Merkle ledger:

- **Integrity** — Each commit has a cryptographic hash chaining to its parent
- **Rollback** — Revert to any previous state via `perspt ledger --rollback`
- **Resume** — `perspt resume` rehydrates session state including energy history, retry counts, and escalation reports
- **Audit** — Complete trail of AI-generated changes with energy breakdowns

```
perspt ledger --recent     # View recent commits
perspt ledger --rollback abc123
perspt ledger --stats      # Session statistics
```

7.1.14 Provisional Branches

When SRBN speculates on child nodes before the parent is fully committed, it uses **provisional branches** to isolate speculative work.

> **Key Invariant**
>
> Provisional work is **never** merged into the global ledger until the parent node meets the stability threshold. If the parent fails, all dependent branches are flushed.

Branch Lifecycle

State	Description
Active	Branch is open, speculative work in progress
Sealed	Parent interface is sealed; children may proceed
Merged	Parent committed; branch work merged into global ledger
Flushed	Parent failed; branch work discarded (may be replayed later)

Interface Seals

Interface nodes produce a structural digest (SHA-256 hash) of their public API after reaching the Commit phase. This seal is injected into child node restriction maps, ensuring children code against stable signatures.

1. Parent node reaches **Commit** phase
2. If the node is an **Interface** class, its exported signatures are hashed
3. `InterfaceSealed` event is emitted with sealed paths and hash
4. Blocked dependents are released
5. Seal digests are available to child verifiers for contract checking

Flush Cascade

When a parent node fails verification:

1. The parent's provisional branch is flushed
2. `collect_descendants` walks the DAG to find all transitive children
3. Each descendant branch is flushed recursively
4. `BranchFlushed` event is emitted with the reason and affected IDs

7.2 Workspace Crates

Perspt is organized as a **9-crate Rust workspace** under the `crates/` directory, plus a meta-crate (`perspt`) that re-exports all libraries.

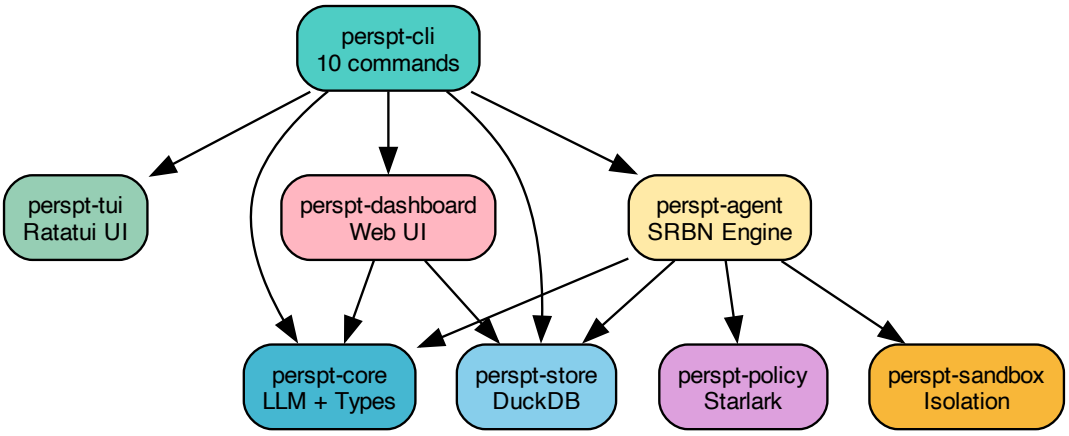


Figure 2: Crate Dependencies

7.2.1 Crate Summary

Crate	Purpose	Key Types
perspt-cli	Binary entry point with 10 subcommands	`Cli`, `Commands`
perspt-core	LLM provider abstraction, config, types, events, plugins	`GenAIProvider`, `Config`, `SRBNNode`, `AgentContext`, `AgentEvent`
perspt-agent	SRBN orchestrator, agents, LSP client, tools, test runner, ledger	`SRBNOrchestrator`, `Agent` trait, `LspClient`, `AgentTools`, `MerkleLedger`
perspt-tui	Ratatui-based terminal UI for chat and agent monitoring	`ChatApp`, `AgentApp`, `DiffViewer`, `ReviewModal`, `TaskTree`
perspt-store	DuckDB persistence for sessions, energy, LLM logs	`SessionStore`, `EnergyRecord`, `LlmRequestRecord`
perspt-policy	Starlark-based policy engine and command sanitization	`PolicyEngine`, `sanitize_command`, `SanitizeResult`
perspt-sandbox	Sandboxed command execution with resource limits	`SandboxedCommand`, `CommandResult`
perspt-dashboard	Browser-based agent monitoring (Axum + Askama + HTMX)	`build_router`, `AppState`, SSE stream

perspt-cli

The CLI crate parses arguments with `clap` and dispatches to the appropriate handler. Subcommands: `chat`, `agent`, `simple-chat`, `init`, `config`, `ledger`, `status`, `abort`, `resume`, `logs`.

perspt-core

Contains all shared types used across the workspace:

- **Types**: `SRBNNode`, `NodeState`, `NodeClass`, `ModelTier`, `TaskPlan`, `BehavioralContract`, `StabilityMonitor`, `EnergyComponents`, `AgentContext`, `TokenBudget`, `RetryPolicy`, `OwnershipManifest`, `PlanningPolicy`, `FeatureCharter`, `BudgetEnvelope`, `RepairFootprint`, `SessionOutcome`, `NodeOutcome`

- **Events**: `AgentEvent` enum with 40+ lifecycle event variants (PSP-5)
- **Plugins**: Language plugin registry (Rust, Python, JS, Go) with LSP config, test runner, init commands, and required binaries
- **Config**: `Config` struct with provider, model, and API key
- **LLM Provider**: `GenAIProvider` — thread-safe wrapper around the `genai` crate with streaming support and retry logic

perspt-agent

The heart of SRBN:

- **Orchestrator**: `SRBNOrchestrator` manages the DAG (`petgraph`), drives the 7-phase control loop, manages LSP clients, and integrates TUI events. Split into submodules: `mod.rs`, `bundle.rs`, `commit.rs`, `convergence.rs`, `init.rs`, `planning.rs`, `repair.rs`, `solo.rs`, `verification.rs`
- **Agents**: `Agent` trait with four implementations — `ArchitectAgent`, `ActuatorAgent`, `VerifierAgent`, `SpeculatorAgent`
- **Tools**: `AgentTools` — `read_file`, `write_file`, `apply_patch`, `apply_diff`, `run_command`, `search_code`, `list_files`, `sed_replace`, `awk_filter`, `diff_files`
- **LSP Client**: Native LSP client supporting `rust-analyzer`, `ty`, `pyright`, `typescript-language-server`, `gopls`
- **Test Runner**: `PythonTestRunner` (pytest) with V_log calculation
- **Ledger**: `MerkleLedger` backed by `perspt-store`
- **Context Retriever**: `ripgrep`-based code search for context injection
- **Prompts**: Centralized prompt templates in `prompts.rs` with constants and `render_*` helper functions for all agent tiers

perspt-tui

Ratatui components:

- **ChatApp** — Interactive chat with markdown rendering and virtual scrolling
- **AgentApp** — SRBN monitoring with dashboard, task tree, and diff viewer
- **ReviewModal** — Approval UI with grouped diff view and verification gates
- **LogsViewer** — LLM request/response log inspector
- **Dashboard** — Status display with energy breakdown
- **TaskTree** — Node execution tree with state indicators

perspt-store

DuckDB-based persistence layer. Tables include sessions, node states, energy records, verification results, LLM request logs, provisional branches, interface seals, artifact bundles, escalation reports, and sheaf validations.

perspt-policy

Starlark-based execution policy:

- `sanitize_command()` validates commands before execution
- `validate_workspace_bound()` ensures file operations stay within the project
- `validate_artifact_mutation()` protects root project files from delete/move

perspt-sandbox

Sandboxed command execution with resource limits. Used for running build and test commands in controlled environments during provisional branch verification.

perspt-dashboard

Browser-based real-time monitoring of agent sessions. Built with Axum 0.8, Askama 0.15 templates, HTMX 2 for partial updates, and Tailwind v4 / DaisyUI 5 for styling. Opens the session store in read-only mode so it never interferes with the running agent. Provides Overview, DAG, Energy, LLM, Sandbox, and Decisions pages plus an SSE stream for live updates.

7.3 PSP Process

Perspt Specification Proposals (PSPs) are design documents that describe significant features, architectural changes, or process improvements.

7.3.1 Purpose

PSPs serve as the primary mechanism for:

- Proposing major features before implementation
- Documenting design decisions and trade-offs
- Providing a historical record of architectural evolution
- Enabling community review of significant changes

7.3.2 PSP Lifecycle

Status	Meaning
Draft	Proposal is being written
Active	Proposal is accepted and under implementation
Final	Implementation is complete
Superseded	Replaced by a newer PSP

7.3.3 Key PSPs

PSP	Title	Status
PSP-1	Core Chat Interface	Final
PSP-2	Multi-Provider Support	Final
PSP-3	Simple CLI Mode	Final
PSP-4	SRBN Agent Mode	Superseded by PSP-5
PSP-5	**Multi-File Coding UX and Repo-Native Verification**	**Final**
PSP-6	Web Dashboard and Real-Time Monitoring	Final
PSP-7	**Robust Typed Correction Loops and Plugin-Aware Prompt Contracts**	**Active**

PSP-5: The Core Lifecycle

PSP-5 is the operative specification that governs the current SRBN agent runtime. It supersedes PSP-4 and introduces:

- **Project-first execution** — multi-file DAGs instead of single-file tasks
- **Ownership closure** — each file owned by exactly one node
- **Artifact bundle protocol** — structured JSON with write/diff/command operations
- **Node classes** — Interface, Implementation, Integration
- **Five-component energy** — V_syn, V_str, V_log, V_boot, V_sheaf

- **Plugin-driven verification** — language plugins select LSP, test runner, init commands
- **Provisional branches** — speculative child execution isolated until parent commits
- **Interface seals** — SHA-256 digest of exported signatures for dependency management
- **Deterministic fallback planner** — handles plan parsing failures gracefully
- **Ledger-based resume** — trustworthy session continuation from any interruption point

See the full specification at `docs/psps/source/psp-000005.rst`.

PSP-7: The Current Runtime

PSP-7 is the operative specification that governs the current SRBN agent runtime. It builds on PSP-5's foundation and introduces:

- **Typed parse pipeline** — 5-layer fail-closed parsing (raw capture → strict JSON → tolerant recovery → schema validation → semantic filtering) replacing Option-based extraction
- **Retry classification** — `RetryClassification` enum (MalformedRetry, Retarget, SupportFileViolation, Replan) enabling intelligent convergence loop decisions
- **Prompt compiler with provenance** — Structured `PromptEvidence` replaces ad-hoc template constants; exact target paths from evidence included in correction prompts
- **Structured JSON artifact format** — `{ artifacts: [], commands: [] }` schema replaces free-form `File: ...` output instructions
- **Manifest policy enforcement** — Semantic validation prevents implicit mutation of root manifests unless explicitly listed as output targets
- **Strict budget exhaustion** — `any_exhausted()` checks all budget dimensions (steps, revisions, cost) before LLM calls
- **Correction attempt records** — Every correction attempt persisted with parse state, retry classification, energy snapshot, and response fingerprint for full observability

PSP-5 remains the foundation for the core SRBN lifecycle. See the full specification at `docs/psps/source/psp-000005.rst`.

See the PSP-7 specification at `docs/psps/source/psp-000007.rst`.

7.3.4 Writing a PSP

1. Fork the repository and create a branch
2. Copy the PSP template from `docs/psps/source/`
3. Fill in the sections: Abstract, Motivation, Specification, Rationale, Reference
4. Submit a pull request for community review
5. Iterate based on feedback

Chapter 8

How-To Guides

Task-oriented recipes for common operations.

Agent Options Full reference for `perspt agent` flags.

Agent Options Reference Configuration Config files, env vars, and precedence.

Configuration Providers Set up each LLM provider.

Set Up Providers Security and Policy Rules Starlark policies, sandboxing, and security.

Security and Policy Rules Dashboard Setup Configure and launch the web dashboard.

Dashboard Setup

8.1 Agent Options Reference

Complete reference for `perspt agent` flags.

8.1.1 Basic Options

Flag	Description
`<TASK>`	Task description (positional argument)
`-w, --workdir <DIR>`	Working directory for code generation
`-y, --yes`	Auto-approve all changes (headless mode)
`--model <MODEL>`	LLM model for all tiers
`--single-file`	Single-file mode (no DAG planning)

8.1.2 Per-Tier Model Selection

Flag	Description
`--architect-model <M>`	Model for task decomposition and planning
`--actuator-model <M>`	Model for code generation
`--verifier-model <M>`	Model for stability analysis
`--speculator-model <M>`	Model for branch prediction
`--architect-fallback-model <M>`	Fallback for architect tier
`--actuator-fallback-model <M>`	Fallback for actuator tier
`--verifier-fallback-model <M>`	Fallback for verifier tier
`--speculator-fallback-model <M>`	Fallback for speculator tier

8.1.3 Energy and Convergence

Flag	Description
`--energy-weights <W>`	Comma-separated `alpha,beta,gamma` (default: `1.0,0.5,2.0`)
`--stability-threshold <E>`	Convergence epsilon (default: `0.10`)
`--verifier-strictness <S>`	`default`, `strict`, or `minimal`

8.1.4 Cost and Limits

Flag	Description
`--max-cost <USD>`	Maximum total LLM spend
`--max-steps <N>`	Maximum total iterations across all nodes

8.1.5 Execution Control

Flag	Description
`--mode <MODE>`	`cautious`, `balanced`, or `yolo`
`-k <N>`	Complexity threshold for balanced mode
`--complexity <LEVEL>`	`low`, `medium`, `high`, `critical`
`--defer-tests`	Skip V_log during node coding
`--auto-approve-safe`	Auto-approve nodes below complexity threshold

8.1.6 Logging and Output

Flag	Description
`--log-llm`	Log all LLM calls to DuckDB
`--output-plan <FILE>`	Export task plan as JSON before execution

8.1.7 Embedded Dashboard

Flag	Description
`--dashboard`	Start the web monitoring dashboard alongside the agent
`--dashboard-port <PORT>`	Port for the embedded dashboard server (default: `3000`)

When `--dashboard` is provided, an Axum web server is spawned as a background task within the agent process. It opens a separate read-only DuckDB connection to the same database file the agent writes to—DuckDB supports one writer plus concurrent readers—so the dashboard can display live progress without interfering with the agent.

The dashboard server is automatically stopped when the agent process exits. See *Dashboard Setup* for dashboard configuration details.

8.1.8 Examples

```
# Simple headless run
perspt agent -y -w /tmp/proj "Create a Python calculator"

# Full control
perspt agent \
  --architect-model gemini-pro-latest \
  --actuator-model gemini-3.1-flash-lite-preview \
  --verifier-model gemini-pro-latest \
  --energy-weights "1.0,1.0,2.0" \
  --stability-threshold 0.05 \
  --max-cost 5.0 \
  --max-steps 30 \
  --log-llm \
  --mode balanced \
  -w ./project "Build a web server"

# Run agent with live dashboard
perspt agent --dashboard "Refactor the auth module"

# Agent with dashboard on custom port
perspt agent --dashboard --dashboard-port 8080 -w ./myapp "Add unit tests"
```

8.2 Configuration

8.2.1 Config File Location

Perspt searches for configuration in this order:

1. `--config <path>` (CLI flag)
2. `~/.config/perspt/config.json`
3. Environment variables
4. Auto-detection

8.2.2 Config File Format

```
{
  "default_provider": "gemini",
  "default_model": "gemini-3.1-flash-lite-preview",
  "api_key": "AIza...",
  "provider_type": "gemini",
  "providers": {
    "openai": {
      "api_key": "sk-xxx",
      "default_model": "gpt-4.1"
    },
    "anthropic": {
      "api_key": "sk-ant-xxx",
      "default_model": "claude-sonnet-4-20250514"
    }
  }
}
```

8.2.3 Environment Variables

Variable	Provider	Priority
`ANTHROPIC_API_KEY`	Anthropic	Highest
`OPENAI_API_KEY`	OpenAI	2
`GEMINI_API_KEY`	Gemini	3
`GROQ_API_KEY`	Groq	4
`COHERE_API_KEY`	Cohere	5
`XAI_API_KEY`	xAI	6
`DEEPSEEK_API_KEY`	DeepSeek	7
(none)	Ollama	Fallback

Note

When multiple keys are set, the highest-priority provider is used. Override with `--provider-type`.

8.2.4 CLI Flag Override

CLI flags always take precedence:

```
# Override provider
perspt chat --provider-type openai --model gpt-4.1

# Override API key
perspt chat --api-key "sk-xxx"

# Use a specific config file
perspt chat --config /path/to/config.json
```

8.2.5 Logging Configuration

```
# Default: error-level logging only (avoids TUI noise)
perspt

# Enable debug logging with RUST_LOG
RUST_LOG=debug perspt simple-chat

# Agent LLM logging to DuckDB
perspt agent --log-llm -w . "Task"
```

8.3 Set Up Providers

8.3.1 OpenAI

```
export OPENAI_API_KEY="sk-xxx"
perspt chat --model gpt-4.1
```

Supported models: `gpt-4.1`, `gpt-4.1-mini`, `gpt-4.1-nano`, `o4-mini`, `o3`, and others listed by `perspt --list-models`.

8.3.2 Anthropic

```
export ANTHROPIC_API_KEY="sk-ant-xxx"
perspt chat --model claude-sonnet-4-20250514
```

Supported models: `claude-sonnet-4-20250514`, `claude-opus-4-20250514` and others.

8.3.3 Google Gemini

```
export GEMINI_API_KEY="AIza..."
perspt chat --model gemini-3.1-flash-lite-preview
```

Supported models: `gemini-pro-latest`, `gemini-3.1-flash-lite-preview`, `gemini-2.0-flash`, and others.

8.3.4 Groq

```
export GROQ_API_KEY="gsk_xxx"
perspt chat --model llama-3.3-70b-versatile
```

Groq provides ultra-fast inference for open-source models.

8.3.5 Cohere

```
export COHERE_API_KEY="xxx"
perspt chat --model command-r-plus
```

8.3.6 xAI

```
export XAI_API_KEY="xxx"
perspt chat --model grok-3-mini-fast
```

8.3.7 DeepSeek

```
export DEEPSEEK_API_KEY="xxx"
perspt chat --model deepseek-chat
```

8.3.8 Ollama (Local)

No API key needed. Ollama is the fallback when no cloud keys are set.

```
# Start Ollama
ollama serve

# Pull a model
ollama pull llama3.2

# Use with Perspt
perspt chat --model llama3.2
```

Multiple concurrent models are supported. Use `ollama list` to see installed models.

8.4 Security and Policy Rules

Perspt provides multiple layers of security for agent mode.

8.4.1 Starlark Policies (perspt-policy)

The `perspt-policy` crate evaluates Starlark scripts to enforce rules on agent actions. Policies can:

- **Allow/deny file operations** — Prevent writes to sensitive paths
- **Restrict shell commands** — Block dangerous commands (`rm -rf`, etc.)
- **Enforce naming conventions** — Require files match patterns
- **Limit resource usage** — Cap file sizes, directory depth

```
# Example Starlark policy
def check_file_write(path, content):
    if path.startswith("/etc/") or path.startswith("/usr/"):
        return deny("Cannot write to system directories")
    if len(content) > 1_000_000:
        return deny("File too large (>1MB)")
    return allow()

def check_command(cmd):
    forbidden = ["rm -rf", "sudo", "chmod 777"]
    for f in forbidden:
        if f in cmd:
            return deny("Forbidden command: " + f)
    return allow()
```

8.4.2 Sandbox Isolation (perspt-sandbox)

The `perspt-sandbox` crate provides filesystem and process isolation for agent-executed commands:

- **Filesystem scoping** — Commands run in a restricted view of the filesystem
- **Process limits** — Timeout, memory, and CPU constraints
- **Network control** — Optional network access restriction

Configuration:

```
# Agent with sandbox enabled
perspt agent -w ./project "Task"

# The sandbox restricts commands to the working directory
```

8.4.3 Ownership Closure

The PSP-5 ownership closure rule ensures:

- Each file is owned by exactly one DAG node
- No two nodes can write to the same file
- Violations trigger a re-plan by the Architect

This prevents conflicting edits and provides a clear audit trail.

8.4.4 Review Modal

In interactive mode (without `--yes`), every node's changes must be manually approved. The review modal shows:

- Full diff of all changes
- Verification results (V_syn, V_str, V_log, V_boot, V_sheaf)
- Options to reject, correct, or edit

For security-sensitive projects, always use interactive mode.

8.4.5 Merkle Ledger

Every committed change is recorded in a content-addressed Merkle tree stored in DuckDB. This provides:

- **Tamper detection** — Hash chain integrity
- **Full auditability** — Every node's input/output is recorded
- **Rollback capability** — Restore to any point in the session

```
perspt ledger --recent
perspt ledger --stats
```

8.4.6 Best Practices

1. **Use interactive mode for production code** — Always review diffs
2. **Set cost limits** — `--max-cost` prevents runaway spending
3. **Use workspace directories** — `-w <dir>` scopes agent writes
4. **Enable LLM logging** — `--log-llm` for post-run auditing
5. **Review ledger after headless runs** — `perspt ledger --recent`

8.5 Dashboard Setup

Configure and launch the Perspt web dashboard for monitoring agent sessions.

8.5.1 Locate the Config File

The Perspt configuration file lives in your platform's config directory:

- **Linux**: `~/.config/perspt/config.toml`
- **macOS**: `~/Library/Application Support/perspt/config.toml`

If migrating from the legacy `~/.perspt/` layout, Perspt will warn and fall back to the old location automatically.

8.5.2 Configure a Dashboard Password

Add to `config.toml`:

```
[dashboard]
password = "your-secret"
```

Without this section, the dashboard runs in open-access mode (suitable for localhost-only use).

8.5.3 Change the Default Port

The default port is `3000`. Override via CLI flag:

```
perspt dashboard --port 8080
```

8.5.4 Build from Source

```
cargo build -p perspt-dashboard --release
```

The dashboard binary is built as part of the `perspt` CLI. Running `cargo build --release` from the workspace root includes it.

8.5.5 Launch Locally

```
perspt dashboard
```

The server binds to `127.0.0.1:3000` by default. Open `http://localhost:3000` in your browser.

8.5.6 Point to a Specific Database

By default, the dashboard reads from the platform data directory (`~/.local/share/perspt/perspt.db` on Linux). To use a different database file:

```
perspt dashboard --db-path /path/to/perspt.db
```

8.5.7 Embed in Agent Mode

Instead of running the dashboard as a separate process, you can start it alongside the agent using the `--dashboard` flag:

```
perspt agent --dashboard "Create a REST API"
```

This spawns the dashboard as a background task within the agent process. It opens a separate read-only DuckDB connection to the same database file the agent writes to — DuckDB supports one writer plus concurrent readers.

To use a custom port:

```
perspt agent --dashboard --dashboard-port 8080 "Add tests"
```

The embedded dashboard provides the same interface as the standalone `perspt dashboard` command: DAG topology, energy convergence, LLM telemetry, and correction-attempt provenance. It stops automatically when the agent process exits.

> **Note**
>
> The embedded dashboard is especially useful for headless CI runs (`--yes --dashboard`) where you want a browser view of progress without a separate terminal.

Chapter 9

Configuration Guide

Perspt supports zero-config auto-detection, environment variables, a JSON config file, and command-line flags. They are applied in this priority order (highest first):

1. **Command-line arguments**
2. **Configuration file** (`config.json`)
3. **Environment variables**
4. **Auto provider detection**
5. **Built-in defaults**

9.1 Automatic Provider Detection

Set any supported API key environment variable and run `perspt` with no arguments:

Priority	Provider	Environment Variable	Default Model
1	OpenAI	`OPENAI_API_KEY`	`gpt-4o-mini`
2	Anthropic	`ANTHROPIC_API_KEY`	`claude-sonnet-4-20250514`
3	Google Gemini	`GEMINI_API_KEY`	`gemini-3.1-flash-lite-preview`
4	Groq	`GROQ_API_KEY`	`llama-3.1-70b-versatile`
5	Cohere	`COHERE_API_KEY`	`command-r-plus`
6	XAI	`XAI_API_KEY`	`grok-beta`
7	DeepSeek	`DEEPSEEK_API_KEY`	`deepseek-chat`
8	AWS Bedrock	`AWS_ACCESS_KEY_ID` (and region/creds)	`us.amazon.nova-pro-v1:0`
9	Google Agent Platform	`VERTEX_API_KEY` (and project/region)	`gemini-2.5-flash`
10	Ollama	*(none — auto-detected)*	`llama3.2`

```
# Example: set a key and run
export GEMINI_API_KEY="your-key"
perspt                # auto-detects Gemini, uses gemini-3.1-flash-lite-preview
perspt chat --model gemini-pro-latest   # override model
```

9.2 Advanced Enterprise Provider Configurations

Unlike standard API-key based providers, enterprise platforms like **AWS Bedrock** and **Google Agent Platform (formerly Vertex AI)** require multi-part configurations and secure credentials to function.

9.2.1 AWS Bedrock (SigV4 Authentication)

Perspt integrates natively with AWS Bedrock via the AWS Signature Version 4 (SigV4) protocol. It automatically detects your AWS configuration from standard AWS environment variables or your local AWS config files.

Required Environment Variables:

Variable	Description
`AWS_ACCESS_KEY_ID`	Your AWS Access Key ID.
`AWS_SECRET_ACCESS_KEY`	Your AWS Secret Access Key.
`AWS_REGION`	The AWS region hosting Bedrock (e.g., `us-east-1` or `us-west-2`).
`AWS_SESSION_TOKEN`	*(Optional)* Required if using temporary AWS IAM credentials.

Local Profile Configuration:

If environment variables are not set, Perspt will automatically read credentials from your local profile (e.g., `~/.aws/credentials` and `~/.aws/config`) using the standard AWS resolution chain:

```
# ~/.aws/config
[default]
region = us-east-1

# ~/.aws/credentials
[default]
aws_access_key_id = AKIAIOSFODNN7EXAMPLE
aws_secret_access_key = wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY
```

9.2.2 Google Agent Platform (formerly Vertex AI)

Google Agent Platform/Vertex AI uses secure OAuth2 Bearer Tokens rather than standard static API keys. You must supply your Google Cloud Project ID and regional location alongside the access token.

Required Environment Variables:

Variable	Description
`VERTEX_API_KEY`	Your OAuth2 Bearer access token (generate dynamically via gcloud CLI).
`VERTEX_PROJECT_ID`	Your Google Cloud Platform (GCP) Project ID.
`VERTEX_LOCATION`	The GCP region hosting Vertex AI resources (e.g., `us-central1` or `europe-west3`).

Token Generation Quickstart:

Since OAuth2 access tokens are short-lived (usually expiring in 1 hour), you can export the token dynamically in your shell before running Perspt:

```
# 1. Authenticate with Google Cloud CLI
gcloud auth login
```

(continues on next page)

(continued from previous page)

```
# 2. Configure variables and inject your access token
export VERTEX_PROJECT_ID="your-gcp-project-123"
export VERTEX_LOCATION="us-central1"
export VERTEX_API_KEY=$(gcloud auth print-access-token)

# 3. Launch Perspt using a Vertex AI model
perspt chat --model gemini-2.5-flash
```

9.3 Supported Models & Naming Conventions

Perspt features an intelligent router that resolves your target provider from the model name prefix. In version 0.6.0, we support the latest generation of models across all providers:

Provider	Model Prefix / Name	Target Model ID
OpenAI	`gpt-5.5-*` / `gpt-4o-*`	`gpt-5.5-preview`, `gpt-4o-mini`
Anthropic	`claude-4.7-*` / `claude-3-5-*`	`claude-sonnet-4.7`, `claude-3-5-sonnet-20241022`
Google Gemini	`gemini-3.1-*` / `gemini-3.0-*`	`gemini-3.1-flash`, `gemini-3.1-pro`
AWS Bedrock	`aws.*` / `bedrock.*`	`us.amazon.nova-pro-v1:0`, `us.anthropic.claude-3-5-sonnet-v2:0`
Google Agent Platform	`vertex.*`	`vertex::gemini-2.5-pro`, `vertex::claude-sonnet-4-6`

9.4 Configuration File

Create `config.json` in one of these locations (searched in order):

1. Path given via `perspt --config <PATH>`
2. `./config.json` in current directory
3. `~/.config/perspt/config.json` (Linux), `~/Library/Application Support/perspt/config.json` (macOS)

Minimal example:

```
{
  "api_key": "your-key",
  "default_model": "gemini-pro-latest",
  "default_provider": "gemini",
  "provider_type": "gemini"
}
```

Full example:

```
{
  "api_key": "your-key",
  "default_model": "gemini-pro-latest",
  "default_provider": "gemini",
  "provider_type": "gemini",
  "providers": {
```

(continues on next page)

(continued from previous page)

```
    "openai": "https://api.openai.com/v1",
    "anthropic": "https://api.anthropic.com",
    "google": "https://generativelanguage.googleapis.com/v1beta"
  }
}
```

Note

The `providers` map overrides endpoint URLs. This is useful for Azure OpenAI, proxy servers, or self-hosted endpoints.

9.5 Command-Line Flags

Global flags apply to all subcommands:

Flag	Description
`-v, --verbose`	Enable verbose logging
`-c, --config <PATH>`	Path to configuration file
`-h, --help`	Show help
`-V, --version`	Show version

Chat-specific:

```
perspt chat --model <MODEL>
```

Agent-specific (see *Agent Options Reference* for the full list):

```
perspt agent [OPTIONS] "<TASK>"

# Key options:
--model <MODEL>               # Default model for all tiers
--architect-model <MODEL>     # Architect tier
--actuator-model <MODEL>      # Actuator tier
--verifier-model <MODEL>      # Verifier tier
--speculator-model <MODEL>    # Speculator tier
-w, --workdir <DIR>           # Working directory
-y, --yes                     # Auto-approve (headless)
--defer-tests                 # Skip V_log during coding
--mode <MODE>                 # cautious | balanced | yolo
--max-cost <USD>              # Maximum cost in USD
--max-steps <N>               # Maximum iterations
--energy-weights <a,b,g>      # Custom alpha,beta,gamma
--stability-threshold <e>     # Custom epsilon
--log-llm                     # Log all LLM calls to DB
--single-file                 # Force single-file mode
--verifier-strictness <LVL>   # default | strict | minimal
```

Manage configuration interactively:

```
perspt config --show    # Print current config
perspt config --edit    # Open in $EDITOR
perspt config --set provider_type=gemini
```

Initialize project-level configuration:

```
perspt init --memory --rules
```

9.6 Dashboard Configuration

The `perspt dashboard` subcommand accepts these options:

Flag	Default	Description
`--port`	`3000`	HTTP port for the dashboard server
`--bind`	`127.0.0.1`	Bind address (use `0.0.0.0` for remote access)
`--db-path`	Platform default	Path to the DuckDB database file

The dashboard opens the database in **read-only** mode and never writes to it. When bound to `127.0.0.1`, cookies are set without the `Secure` flag so plain HTTP works on localhost.

Chapter 10

Reference

Precise, authoritative reference for Perspt's interface.

CLI Reference Complete command-line interface documentation.

CLI Reference Key Bindings TUI keyboard shortcuts and review modal controls.

Key Bindings Troubleshooting Advanced diagnostics and known issues.

Advanced Troubleshooting

10.1 CLI Reference

```
perspt [OPTIONS] [COMMAND]
```

10.1.1 Global Options

Flag	Description
`--config <PATH>`	Path to configuration file
`--api-key <KEY>`	API key for the LLM provider
`--provider-type <TYPE>`	Provider: `openai`, `anthropic`, `gemini`, `groq`, `cohere`, `xai`, `deepseek`, `ollama`
`--provider <NAME>`	Provider name (equivalent to `--provider-type`)
`--model <MODEL>`	Model identifier
`--list-models`	List available models and exit
`-h, --help`	Print help
`-V, --version`	Print version

10.1.2 Commands

`chat` (default)

Launch the TUI chat interface.

```
perspt chat [--model MODEL] [--provider-type TYPE]
```

simple-chat

Launch the plain-text CLI chat.

```
perspt simple-chat [--log-file FILE]
```

dashboard

Launch the real-time web monitoring dashboard.

```
perspt dashboard [--port PORT] [--bind ADDR] [--db-path PATH]
```

- `--port` — HTTP port (default `3000`)
- `--bind` — Bind address (default `127.0.0.1`)
- `--db-path` — Path to DuckDB database file (default: platform data directory)

See *Dashboard Setup* for configuration details.

agent

Run the SRBN autonomous coding agent.

```
perspt agent [OPTIONS] <TASK>
```

- `--dashboard` — Start the web monitoring dashboard alongside the agent
- `--dashboard-port <PORT>` — Port for the embedded dashboard (default `3000`)

See *Agent Options Reference* for full agent options.

init

Initialize a new project with Perspt configuration.

```
perspt init [--workdir DIR]
```

config

View or edit Perspt configuration.

```
perspt config [show|edit|reset]
```

ledger

Query the Merkle ledger.

```
perspt ledger [--recent] [--stats] [--node NODE_ID]
```

status

Show current session status.

```
perspt status
```

Displays: per-node lifecycle counts (queued, running, verifying, retrying, completed, failed, escalated), latest energy breakdown, total retry count, recent escalation reports, step timeline summary (per-step-type counts, total step time), and correction attempt summaries (accepted/rejected counts per node).

abort

Abort the current agent session.

```
perspt abort
```

resume

Resume an interrupted session.

```
perspt resume [--last]
```

Displays trust context before resuming: escalation count, last energy state, total retries. The `BudgetEnvelope` (step/cost/revision caps) is restored from the database so limits continue from the interrupted session.

logs

View LLM call logs and token metrics. Full prompt/response text is only available when `--log-llm` was active during the session; basic token usage, latency, and cost data are always recorded.

```
perspt logs [--tui] [--last] [--stats]
```

10.2 Key Bindings

10.2.1 TUI Chat Mode

Key	Action
Enter	Send message
Ctrl+J	Insert newline in input
Page Up	Scroll chat up one page
Page Down	Scroll chat down one page
Ctrl+Up	Scroll up one line
Ctrl+Down	Scroll down one line
Home	Scroll to top
End	Scroll to bottom
Ctrl+C	Cancel current stream
Esc / Ctrl+Q	Quit

10.2.2 Review Modal (Agent Mode)

Key	Action
y	Approve node changes
n	Reject node changes
c	Send correction feedback
e	Open files in editor
d	Toggle diff view
q	Cancel review (abort node)

10.2.3 Input Editor

Key	Action
Left / Right	Move cursor
Ctrl+A	Move to beginning of line
Ctrl+E	Move to end of line
Ctrl+W	Delete word backward
Ctrl+U	Delete to beginning of line
Ctrl+K	Delete to end of line
Backspace	Delete character backward
Delete	Delete character forward

10.3 Advanced Troubleshooting

10.3.1 Diagnostic Commands

```
# Session status with per-node details
perspt status

# LLM call log browser
perspt logs --tui

# Usage statistics
perspt logs --stats

# Ledger integrity check
perspt ledger --stats

# Enable verbose logging
RUST_LOG=debug perspt simple-chat 2>debug.log
```

10.3.2 Common Error Patterns

`ErrorType::ApiKeyMissing`

No API key found for the selected provider. Set the environment variable or pass `--api-key`.

`ErrorType::ModelNotFound`

The model identifier is not recognized by the provider. Use `perspt --list-models` to see available models.

`ErrorType::ConnectionFailed`

Cannot reach the provider endpoint. Check:

- Internet connectivity
- Ollama service status (`ollama serve`)
- Firewall/proxy settings

`ErrorType::RateLimitExceeded`

Provider rate limit hit. Wait and retry, or switch providers.

`ErrorType::BudgetExceeded`

LLM spend exceeded `--max-cost` limit. Increase the limit or reduce task scope.

10.3.3 Agent-Specific Issues

Ownership conflict:

Two nodes attempted to write the same file. The Architect replans the DAG to resolve the conflict. If this persists, simplify the task description.

Sheaf validation failure:

Cross-node contracts are incompatible. Common causes:

1. Inconsistent function signatures across modules
2. Missing imports between files
3. Type mismatches at module boundaries

The agent automatically retries with feedback from the sheaf validator.

Degraded verification mode:

Missing tool binaries cause fallback to heuristic verification:

Component	Full Mode	Degraded Mode
V_syn	LSP diagnostics (ty, rust-analyzer)	Regex pattern matching
V_log	Test runner (pytest, cargo test)	Exit code stubs
V_boot	Init commands (uv init, cargo init)	Skipped

Install the required tools to restore full verification.

10.3.4 Terminal Restoration

If Perspt crashes and leaves the terminal in raw mode:

```
reset
# or
stty sane
```

Perspt installs a panic hook that attempts to restore the terminal automatically (raw mode off, leave alternate screen). If this fails, `reset` will fix it.

10.3.5 Performance

Slow response streaming:

1. Check network latency to the provider
2. Try a faster model (e.g., `gemini-3.1-flash-lite-preview`)
3. Use Groq for the fastest inference

High memory usage in agent mode:

1. Large DAGs with many nodes consume more memory
2. Use `--single-file` for simple tasks

3. Use `--max-steps` to bound total iterations

Chapter 11

API Reference

Crate-level API documentation for Perspt's Rust workspace.

> **Tip**
>
> For full Rustdoc-generated documentation, run:
>
> ```
> cargo doc --open --no-deps --all-features
> ```

perspt-core Types, config, LLM provider, events, plugins.

perspt-core perspt-agent SRBN orchestrator, agents, ledger, tools.

perspt-agent perspt-tui Ratatui terminal UI (chat + agent).

perspt-tui perspt-cli Clap CLI entry point and subcommands.

perspt-cli perspt-store DuckDB session persistence.

perspt-store perspt-policy Starlark policy engine.

perspt-policy perspt-sandbox Command sandboxing and isolation.

perspt-sandbox perspt-dashboard Axum + Askama + HTMX web dashboard.

perspt-dashboard

11.1 `perspt-core`

The foundation crate providing all canonical types, configuration, LLM provider integration, event system, and language plugins.

11.1.1 Modules

Module	Description
`types`	All PSP-5 types: SRBNNode, NodeState, NodeClass, ModelTier, EnergyComponents, BehavioralContract, StabilityMonitor, RetryPolicy, TaskPlan, AgentContext, TokenBudget, ArtifactBundle, OwnershipManifest, VerificationResult, EscalationReport, SheafValidationResult, ProvisionalBranch, InterfaceSealRecord, ContextPackage, ContextProvenance, StructuralDigest, SummaryDigest, RestrictionMap, BlockedDependency
`config`	`Config { provider, model, api_key }`
`events`	`AgentEvent` (~30 variants), `AgentAction`, `NodeStatus`, `ActionType`, channel types
`llm_provider`	`GenAIProvider`, `LlmResponse`, `EOT_SIGNAL`, streaming support
`plugin`	`LanguagePlugin` trait, `PythonPlugin`, `RustPlugin`, `JsPlugin`, `PluginRegistry`
`memory`	`ProjectMemory` from `.perspt/memory.toml`
`normalize`	Model and provider name normalization

11.1.2 Key Types

SRBNNode — The core DAG node:

```rust
pub struct SRBNNode {
    pub node_id: String,
    pub goal: String,
    pub context_files: Vec<PathBuf>,
    pub output_targets: Vec<PathBuf>,
    pub contract: BehavioralContract,
    pub tier: ModelTier,
    pub monitor: StabilityMonitor,
    pub state: NodeState,
    pub parent_id: Option<String>,
    pub children: Vec<String>,
    pub node_class: NodeClass,
    pub owner_plugin: Option<String>,
    pub provisional_branch_id: Option<String>,
    pub interface_seal_hash: Option<String>,
}
```

EnergyComponents — Lyapunov energy decomposition:

```rust
pub struct EnergyComponents {
    pub v_syn: f32,    // LSP diagnostics
    pub v_str: f32,    // Contract compliance
    pub v_log: f32,    // Test results
    pub v_boot: f32,   // Bootstrap commands
    pub v_sheaf: f32,  // Cross-node validation
}
```

AgentEvent — 30+ lifecycle events:

`TaskStatusChanged`, `PlanGenerated`, `PlanReady`, `NodeSelected`, `BundleApplied`, `VerificationComplete`, `SheafValidationComplete`, `BranchCreated`, `InterfaceSealed`, `BranchFlushed`, `BranchMerged`, `EscalationClassified`, `GraphRewriteApplied`, `DegradedVerification`, `SensorFallback`, `ContextDegraded`, `ContextBlocked`, `ProvenanceDrift`, `ModelFallback`, `ToolReadiness`, `ApprovalRequest`, `EnergyUpdated`, `NodeCompleted`, `Complete`, `Error`, `Log`, `FallbackPlanner`, `DependentUnblocked`, `StructuralDependencyMissing`

See *Architecture* for the complete type inventory.

11.2 perspt-agent

The SRBN orchestrator and all supporting subsystems.

11.2.1 Modules

Module	Description
`orchestrator`	`SRBNOrchestrator` — petgraph-based DAG execution with PSP-5 lifecycle
`agent`	`Agent` trait + `ArchitectAgent`, `ActuatorAgent`, `VerifierAgent`, `SpeculatorAgent`
`tools`	`AgentTools` — 10+ filesystem and shell tools (read_file, write_file, apply_diff, run_command, search_code, list_files, apply_patch, sed_replace, awk_filter, diff_files)
`ledger`	`MerkleLedger` — Content-addressed commit tracking over DuckDB
`lsp`	`LspClient` — JSON-RPC stdio client for rust-analyzer, ty, pyright, etc.
`test_runner`	`TestRunnerTrait` + `PythonTestRunner`, `RustTestRunner`, `PluginVerifierRunner`
`context_retriever`	`ContextRetriever` — Workspace search with byte budget limits

11.2.2 Key Traits

Agent — Per-tier LLM interaction:

```rust
#[async_trait]
pub trait Agent: Send + Sync {
    async fn process(&self, node: &SRBNNode, ctx: &AgentContext)
        -> Result<AgentMessage>;
    fn name(&self) -> &str;
    fn can_handle(&self, node: &SRBNNode) -> bool;
    fn model(&self) -> &str;
    fn build_prompt(&self, node: &SRBNNode, ctx: &AgentContext) -> String;
}
```

TestRunnerTrait — Plugin-driven verification:

```rust
#[async_trait]
pub trait TestRunnerTrait: Send + Sync {
    async fn run_syntax_check(&self) -> Result<TestResults>;
    async fn run_tests(&self) -> Result<TestResults>;
    async fn run_build_check(&self) -> Result<TestResults>;
    async fn run_lint(&self) -> Result<TestResults>;
    async fn run_stage(&self, stage: VerifierStage) -> Result<TestResults>;
    fn name(&self) -> &str;
}
```

11.2.3 Ledger Types

Type	Description
`MerkleCommit`	commit_id, session_id, node_id, merkle_root, parent_hash, energy, stable
`NodeCommitPayload`	Snapshot of node state for ledger commit
`LedgerStats`	total_sessions, total_commits, db_size_bytes
`NodeReviewSummary`	Energy history, escalations, seals, provenance for a single node
`SessionReviewSummary`	Aggregate stats: total/completed/failed/escalated, branches, review outcomes
`SessionSnapshot`	Full session state for resume: node_details, edges, branches, escalations

11.3 `perspt-tui`

Ratatui-based terminal user interface with two application modes.

11.3.1 Applications

Type	Description
`ChatApp`	Interactive chat with markdown rendering and response streaming
`AgentApp`	Agent dashboard with DAG tree, energy display, and review modal

11.3.2 Widgets

Widget	Description
`Dashboard`	Main agent dashboard layout with panels
`TaskTree`	DAG visualization showing node states and energy
`ReviewModal`	Grouped diff viewer with approve/reject/correct controls
`DiffViewer`	Unified diff display with syntax highlighting
`LogsViewer`	LLM call log browser with filtering
`Theme`	Color scheme and styling

11.3.3 Entry Points

```
pub fn run_chat_tui(...) -> Result<()>;
pub fn run_agent_tui_with_orchestrator(...) -> Result<()>;
pub fn run_logs_viewer(...) -> Result<()>;
pub fn init_terminal() -> Result<TuiTerminal>;
pub fn restore_terminal(terminal: TuiTerminal) -> Result<()>;
```

11.3.4 Channels

```
pub fn create_app_event_channel() -> (AppEventSender, AppEventReceiver);
pub fn create_telemetry_channel() -> (TelemetrySender, TelemetryReceiver);
```

11.4 perspt-cli

Clap-based CLI entry point with 10 subcommands.

11.4.1 Subcommands

Command	Description
`chat`	TUI chat (default)
`simple-chat`	Plain-text streaming chat
`agent`	SRBN autonomous coding agent (40+ options)
`init`	Initialize project configuration
`config`	View/edit/reset configuration
`ledger`	Query Merkle ledger
`status`	Show current session state
`abort`	Abort active session
`resume`	Resume interrupted session
`logs`	View LLM call logs

See *CLI Reference* for the complete flag reference.

11.5 perspt-store

DuckDB-backed session and ledger persistence.

Important

Perspt uses **DuckDB**, not SQLite, for its session store.

11.5.1 Core Type

```rust
pub struct SessionStore {
    conn: Mutex<Connection>,  // duckdb::Connection
}

impl SessionStore {
    pub fn new() -> Result<Self>;           // Default path
    pub fn open(path: &Path) -> Result<Self>; // Custom path
    pub fn open_read_only(path: &Path) -> Result<Self>; // Read-only mode
    pub fn default_db_path() -> PathBuf;    // ~/.local/share/perspt/
}
```

> **Note**
>
> `open_read_only` uses DuckDB's `AccessMode::ReadOnly` and does **not** call `init_schema()`. This makes it safe for concurrent dashboard reads alongside the agent's write lock. The database file must already exist.

11.5.2 Record Types

Type	Description
`SessionRecord`	session_id, task, working_dir, merkle_root, detected_toolchain, status
`NodeStateRecord`	Per-node snapshot (id, state, energy, class, plugin, goal)
`EnergyRecord`	v_syn, v_str, v_log, v_boot, v_sheaf, v_total per node
`LlmRequestRecord`	model, prompt, response, tokens_in/out, latency_ms
`StructuralDigestRecord`	Content hash for interface seals
`ContextProvenanceRecord`	What context each node received
`EscalationReportRecord`	Classified escalation with energy and evidence
`RewriteRecordRow`	Graph rewrite audit trail
`SheafValidationRow`	Cross-node validation results
`ProvisionalBranchRow`	Branch lifecycle tracking
`BranchLineageRow`	Parent-child branch relationships
`InterfaceSealRow`	Sealed interface records
`BranchFlushRow`	Flush cascade records
`TaskGraphEdgeRow`	DAG edges between nodes
`ReviewOutcomeRow`	Human review decisions
`VerificationResultRow`	Full verification snapshots
`ArtifactBundleRow`	Stored artifact bundles per node

11.5.3 DuckDB Tables

The schema is initialized by `init_schema()` and includes 17+ tables matching the record types above.

11.6 `perspt-policy`

Starlark-based policy engine for agent action governance.

11.6.1 Core Types

```rust
pub struct PolicyEngine {
    policies: Vec<FrozenModule>,
    policy_dir: PathBuf,
}

pub enum PolicyDecision {
    Allow,
    Prompt(String),
    Deny(String),
}

pub struct SanitizeResult {
    pub parts: Vec<String>,
```

(continues on next page)

(continued from previous page)

```
    pub warnings: Vec<String>,
    pub rejected: bool,
    pub rejection_reason: Option<String>,
}
```

11.6.2 Functions

Function	Description
`PolicyEngine::new()`	Create engine instance
`PolicyEngine::load_policies()`	Load all .star files from policy directory
`sanitize_command(cmd)`	Parse, validate, and filter a shell command
`validate_workspace_bound(cmd, dir)`	Ensure command stays within working directory
`is_safe_for_auto_exec(cmd)`	Whitelist check for auto-approval in balanced mode

11.7 `perspt-sandbox`

Process isolation for agent-executed commands.

11.7.1 Core Types

```
pub struct CommandResult {
    pub stdout: String,
    pub stderr: String,
    pub exit_code: Option<i32>,
    pub timed_out: bool,
    pub duration: Duration,
}

pub trait SandboxedCommand: Send + Sync {
    fn execute(&self) -> Result<CommandResult>;
    fn display(&self) -> String;
    fn is_read_only(&self) -> bool;
}

pub struct BasicSandbox {
    program: String,
    args: Vec<String>,
    working_dir: Option<PathBuf>,
    timeout: Duration,
}
```

11.7.2 Usage

```
let sandbox = BasicSandbox::new("cargo", vec!["test"])
    .with_working_dir("/path/to/project")
    .with_timeout(Duration::from_secs(60));
```

(continues on next page)

(continued from previous page)

```
let result = sandbox.execute()?;
assert!(result.exit_code == Some(0));
```

`BasicSandbox` can also be created from a command string:

```
let sandbox = BasicSandbox::from_command_string("uv run pytest -v")?;
```

11.8 `perspt-dashboard`

Real-time web dashboard for Perspt agent monitoring, built with Axum + Askama + HTMX.

11.8.1 Core Types

```
pub struct AppState {
    pub store: Arc<SessionStore>,        // read-only DuckDB
    pub password: Option<String>,         // optional auth password
    pub session_token: Arc<Mutex<Option<String>>>,
    pub working_dir: PathBuf,
    pub is_localhost: bool,               // controls Secure cookie flag
}

pub enum DashboardError {
    Store(anyhow::Error),     // 503 — DB unavailable
    Template(askama::Error),  // 500 — render failure
    Internal(String),         // 500 — generic
}
```

`DashboardError` implements `IntoResponse` and renders a styled HTML error page. Store errors return `503 Service Unavailable`.

11.8.2 Router

`build_router(state: AppState) -> Router` constructs the full Axum router:

Method	Route	Handler
GET	`/login`	`auth::login_page` — render login form
POST	`/login`	`auth::login_handler` — validate password, set session cookie
GET	`/`	`handlers::overview::overview_handler` — session list
GET	`/sessions/{session_id}/dag`	`handlers::dag::dag_handler` — DAG topology
GET	`/sessions/{session_id}/energy`	`handlers::energy::energy_handler` — energy convergence
GET	`/sessions/{session_id}/llm`	`handlers::llm::llm_handler` — LLM telemetry
GET	`/sessions/{session_id}/sandbox`	`handlers::sandbox::sandbox_handler` — provisional branches
GET	`/sessions/{session_id}/decisions`	`handlers::decisions::decisions_handler` — decision trace
GET	`/sse/{session_id}`	`sse::sse_handler` — SSE event stream

All routes except `/login` are behind `auth::auth_middleware`. If no password is configured, all requests pass through.

11.8.3 Auth Middleware

Cookie-based authentication with random session tokens:

- On successful login, generates a 32-character alphanumeric token
- Stores token in `AppState::session_token`
- Sets `perspt_session` cookie: `HttpOnly`, `SameSite=Lax`, `Path=/`, `Secure` (when not localhost)
- Middleware checks cookie value against stored token
- No password configured → open access mode

11.8.4 SSE Stream

The SSE endpoint pushes named events every 2 seconds:

- `node-stats` — live summary of node states (total, done, running, failed)

Each event contains an HTML fragment suitable for HTMX `sse-swap`.

11.8.5 Templates

Askama templates live in `crates/perspt-dashboard/templates/`:

- `base.html` — layout with navigation, HTMX, and DaisyUI theme
- `login.html` — login form
- `pages/overview.html` — session list table
- `pages/dag.html` — node cards and edge table
- `pages/energy.html` — energy component table
- `pages/llm.html` — stats bar and request table
- `pages/sandbox.html` — provisional branch table
- `pages/decisions.html` — collapsible sections for escalations, sheaf validations, rewrites, plan revisions, repair footprints, and verification results

11.8.6 View Models

Each page has a corresponding view model in `src/views/`:

- `OverviewViewModel` — sessions, node counts, budgets
- `DagViewModel` — nodes with state/energy, edges
- `EnergyViewModel` — per-node energy components
- `LlmViewModel` — requests with token counts, latency, previews
- `SandboxViewModel` — provisional branches with sandbox directories
- `DecisionsViewModel` — all six decision categories

Chapter 12

Developer Guide

Internals, architecture, and contribution guide for Perspt developers.

Architecture Crate structure, data flow, and PSP-5 internals (including the experimental SRBN engine).

Architecture Contributing Dev setup, coding standards, and PR workflow.

Contributing Extending Perspt Add providers, plugins, tools, and sheaf validators.

Extending Perspt Testing Test infrastructure, patterns, and conventions.

Testing

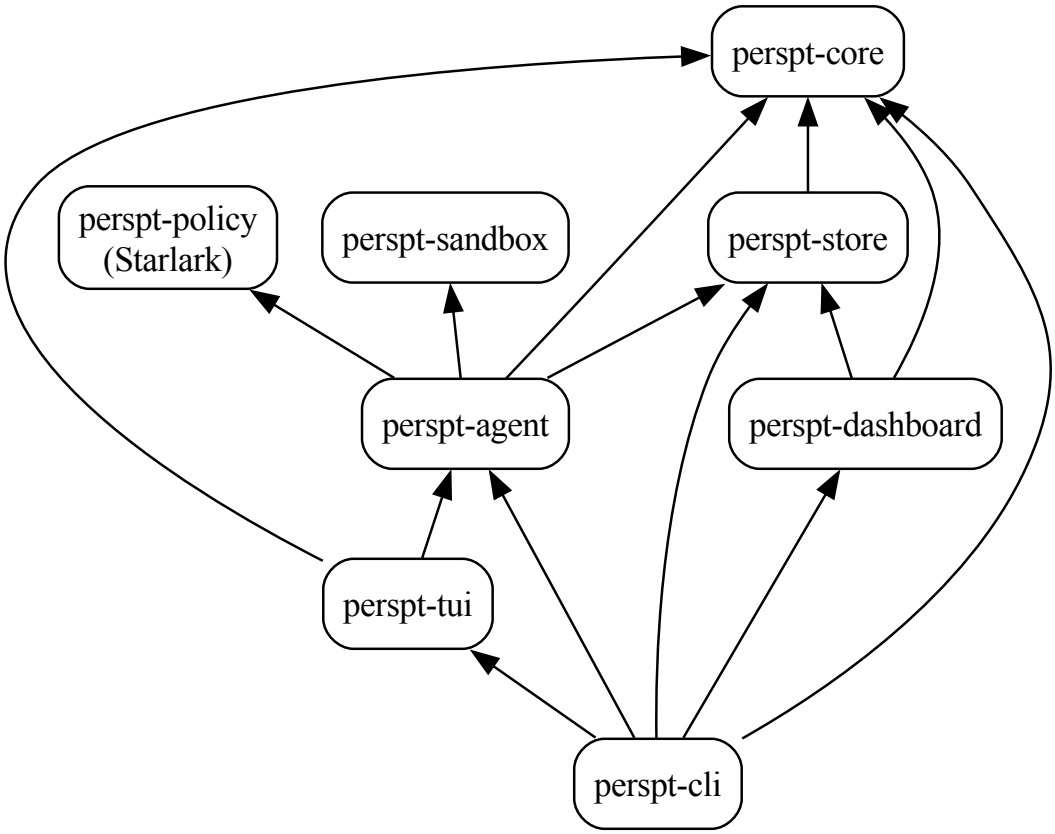
12.1 Architecture

Perspt is a Rust workspace with nine crates including a root integration crate. Version 0.5.9 implements the PSP-7 specification for the experimental SRBN agent.

12.1.1 Workspace Layout

```
perspt/                      # Root: integration crate (perspt)
+-- crates/
|   +-- perspt-core/         # Types, config, LLM provider, events, plugins
|   +-- perspt-agent/        # SRBN orchestrator, agents, ledger, LSP, tools
|   +-- perspt-tui/          # Ratatui TUI (chat + agent + review modal)
|   +-- perspt-cli/          # Clap CLI entry point, subcommands
|   +-- perspt-store/        # DuckDB session store
|   +-- perspt-policy/       # Starlark policy engine
|   +-- perspt-sandbox/      # Command sandboxing
|   +-- perspt-dashboard/    # Axum web dashboard
+-- tests/                   # Integration tests
+-- docs/                    # Sphinx documentation
```

12.1.2 Dependency Graph



12.1.3 Crate: `perspt-core`

The foundation crate. Re-exports all canonical types.

Modules:

- `types` — All PSP-5 types (see *PSP-5 Type Inventory* below)
- `config` — `Config { provider, model, api_key }`
- `events` — `AgentEvent` (~30 variants), `AgentAction`, `NodeStatus`, `ActionType`
- `llm_provider` — `GenAIProvider` wrapping the `genai` crate; `EOT_SIGNAL`
- `plugin` — `LanguagePlugin` trait + `PythonPlugin`, `RustPlugin`, `JsPlugin`
- `memory` — `ProjectMemory` loaded from `.perspt/memory.toml`
- `normalize` — Model and provider name normalization

Key Plugin Types:

```rust
pub trait LanguagePlugin: Send + Sync {
    fn name(&self) -> &str;
    fn detect(&self, path: &Path) -> bool;
    fn get_init_action(&self, opts: &InitOptions) -> ProjectAction;
```

(continues on next page)

(continued from previous page)

```
    fn test_command(&self) -> Option<String>;
    fn syntax_check_command(&self) -> Option<String>;
    fn verifier_profile(&self) -> VerifierProfile;
    fn owns_file(&self, path: &Path) -> bool;
    // ... ~15 methods total
}
```

Plugins provide verifier profiles with fallback chains:

```
pub struct VerifierProfile {
    pub plugin_name: String,
    pub capabilities: Vec<VerifierCapability>,
    pub lsp: LspCapability,
}

pub struct VerifierCapability {
    pub stage: VerifierStage,       // SyntaxCheck | Build | Test | Lint
    pub command: Option<String>,     // Primary command
    pub available: bool,
    pub fallback_command: Option<String>,
    pub fallback_available: bool,
}
```

12.1.4 Crate: `perspt-agent`

The experimental SRBN orchestrator and its subsystems.

Modules:

- `orchestrator` — `SRBNOrchestrator` with `petgraph::DiGraph<SRBNNode, Dependency>`. Split into phase-focused sub-modules: `mod` (struct, constructors, run, helpers, tests), `init` (workspace bootstrap), `solo` (single-file mode), `planning` (architect interaction), `verification` (energy computation), `convergence` (stability loop), `commit` (ledger promotion), `repair` (correction prompts), `bundle` (artifact application).
- `agent` — `Agent` trait + `ArchitectAgent`, `ActuatorAgent`, `VerifierAgent`, `SpeculatorAgent`
- `tools` — `AgentTools` (read_file, write_file, apply_diff, delete_file, move_file, run_command, search_code, etc.)
- `prompts` — Externalized prompt templates for architect (existing/greenfield) and actuator roles
- `ledger` — `MerkleLedger` atop `SessionStore`
- `lsp` — `LspClient` (JSON-RPC over stdio)
- `test_runner` — `TestRunnerTrait` + `PythonTestRunner`, `RustTestRunner`, `PluginVerifierRunner`
- `context_retriever` — `ContextRetriever` for workspace search

Agent Trait:

```
#[async_trait]
pub trait Agent: Send + Sync {
    async fn process(&self, node: &SRBNNode, ctx: &AgentContext)
        -> Result<AgentMessage>;
    fn name(&self) -> &str;
    fn can_handle(&self, node: &SRBNNode) -> bool;
    fn model(&self) -> &str;
    fn build_prompt(&self, node: &SRBNNode, ctx: &AgentContext) -> String;
}
```

Orchestrator:

```rust
pub struct SRBNOrchestrator {
    pub graph: DiGraph<SRBNNode, Dependency>,
    pub context: AgentContext,
    // + ledger, agents (4 tiers), tools, LSP clients, test runners,
    //   event/action channels, per-tier model names + fallbacks
}
```

The orchestrator drives the PSP-5 lifecycle:

1. `detect_workspace()` — Identify plugins and workspace state
2. `plan_task()` — Architect decomposes task into DAG
3. `execute_dag()` — Topological traversal with per-node verification loop
4. `verify_node()` — Compute V(x) via plugin verifier profile
5. `sheaf_validate()` — Cross-node contract checking
6. `review_node()` — Interactive approval (unless `--yes`)
7. `execute_node()` — Returns `NodeOutcome::Completed` when $V(x) \leq \varepsilon$ or `NodeOutcome::Escalated` when retries are exhausted
8. `commit_node()` — Record stable node in Merkle ledger

After all nodes are processed, the orchestrator derives `SessionOutcome` from completed/escalated counts and emits a `Complete` event.

12.1.5 Crate: `perspt-store`

DuckDB-backed persistence. **Not SQLite.**

```rust
pub struct SessionStore {
    conn: Mutex<Connection>,  // duckdb::Connection
}
```

Tables:

- `sessions` — Session metadata (id, task, working_dir, merkle_root, status)
- `node_states` — Per-node snapshots
- `energy_records` — Energy history
- `llm_requests` — Full LLM request/response logging
- `structural_digests` — Content hashes for interface seals
- `context_provenance` — Provenance tracking per node
- `escalation_reports` — Classified escalations
- `rewrite_records` — Graph rewrite audit trail
- `sheaf_validations` — Cross-node validation results
- `provisional_branches` — Branch lifecycle
- `branch_lineage` — Parent-child branch relationships
- `interface_seals` — Sealed interface records
- `branch_flushes` — Flush cascade records
- `task_graph_edges` — DAG edges
- `review_outcomes` — Human review decisions
- `verification_results` — Full verification snapshots
- `artifact_bundles` — Bundle JSON per node
- `plan_revisions` — Plan version history
- `feature_charters` — Scope governance records
- `repair_footprints` — Correction audit trail
- `budget_envelopes` — Session budget caps and usage

12.1.6 Crate: `perspt-tui`

Ratatui-based terminal UI with two modes:

- **ChatApp** — Interactive chat with markdown rendering and streaming
- **AgentApp** — Agent dashboard with DAG tree, energy display, review modal

Key components:

Component	Purpose
`Dashboard`	Main agent dashboard layout
`TaskTree`	DAG visualization with node states
`ReviewModal`	Grouped diff viewer with approve/reject/correct
`DiffViewer`	Unified diff display
`LogsViewer`	LLM log browser
`FrameRateLimiter`	60fps cap, adaptive rendering

12.1.7 Crate: `perspt-policy`

Starlark policy evaluation:

```rust
pub struct PolicyEngine {
    policies: Vec<FrozenModule>,
    policy_dir: PathBuf,
}

pub enum PolicyDecision {
    Allow,
    Prompt(String),
    Deny(String),
}
```

Utility functions:

- `sanitize_command(cmd)` → `SanitizeResult` (split, validate, filter)
- `validate_workspace_bound(cmd, working_dir)` — Ensure commands stay in scope
- `validate_artifact_mutation(path, workspace_root, operation)` — Protect root project files from delete/move

12.1.8 Crate: `perspt-sandbox`

Process isolation with active timeout enforcement:

```rust
pub trait SandboxedCommand: Send + Sync {
    fn execute(&self) -> Result<CommandResult>;
    fn display(&self) -> String;
    fn is_read_only(&self) -> bool;
}

pub struct BasicSandbox {
    program: String,
    args: Vec<String>,
    working_dir: Option<PathBuf>,
    timeout: Duration,  // Active: spawn + poll + kill on deadline
}
```

12.1.9 PSP-5 Type Inventory

All canonical types live in `perspt_core::types`:

SRBN Core:

Type	Description
`SRBNNode`	DAG node: goal, output_targets, contract, tier, monitor, state, node_class, owner_plugin, provisional_branch_id, interface_seal_hash
`NodeState` (12 variants)	TaskQueued → Planning → Coding → Verifying → Retry → SheafCheck → Committing → Completed / Failed / Escalated / Aborted / Superseded. Includes `from_display_str()` (case-insensitive canonical parser with legacy aliases), `is_success()`, `is_active()`, and `Display` impl.
`NodeClass`	Interface, Implementation (default), Integration
`ModelTier`	Architect, Actuator, Verifier, Speculator
`BehavioralContract`	interface_signature, invariants, forbidden_patterns, weighted_tests, energy_weights
`StabilityMonitor`	energy_history, attempt_count, stable, stability_epsilon, max_retries, retry_policy
`RetryPolicy`	Per-error-type counters: compilation, tool, review
`SessionOutcome`	Success (all nodes completed), PartialSuccess (some escalated), Failed (none completed)
`NodeOutcome`	Completed ($V(x) \leq \varepsilon$) or Escalated (retries exhausted). Returned by `execute_node()`

Energy:

Type	Fields
`EnergyComponents`	v_syn (LSP), v_str (contracts), v_log (tests), v_boot (init), v_sheaf (cross-node)
`Criticality`	Critical (10.0), High (3.0), Low (1.0)
`WeightedTest`	test_name, criticality

Task Planning:

Type	Description
`TaskPlan`	Container for `Vec<PlannedTask>`
`PlannedTask`	id, goal, output_files, dependencies, task_type, contract, command_contract, node_class
`TaskType`	Code, Command, UnitTest, IntegrationTest, Refactor, Documentation
`CommandContract`	command, expected_exit_code, expected_files, forbidden_stderr_patterns

Artifact Bundle:

Type	Description
`ArtifactBundle`	artifacts: Vec<ArtifactOperation>, commands: Vec<String>
`ArtifactOperation`	Write { path, content } | Diff { path, patch } | Delete { path } | Move { from, to }
`OwnershipManifest`	entries: HashMap<String, OwnershipEntry>, fanout_limit

Verification:

Type	Description
`VerificationResult`	syntax_ok, build_ok, tests_ok, lint_ok, diagnostics_count, tests_passed/failed, degraded, stage_outcomes
`StageOutcome`	stage, passed, sensor_status, output
`SensorStatus`	Available | Fallback { actual, reason } | Unavailable { reason }
`VerifierStrictness`	Default, Strict, Minimal

Context Management:

Type	Description
`AgentContext`	working_dir, history, merkle_root, complexity_k, session_id, auto_approve, defer_tests, token_budget, execution_mode, active_plugins, ownership_manifest
`TokenBudget`	max_tokens, max_cost_usd, tokens_used (in/out), cost_usd, per-1k rates
`ContextBudget`	byte_limit (100KB), file_count_limit (20)
`RestrictionMap`	Per-node context scoping: owned_files, sealed_interfaces, structural_digests
`ContextPackage`	Assembled context with budget tracking
`ContextProvenance`	Audit trail of what context each node received

Escalation and Rewrite:

Type	Description
`EscalationCategory`	ImplementationError, ContractMismatch, InsufficientModelCapability, DegradedSensors, TopologyMismatch
`RewriteAction` (9 variants)	GroundedRetry, ContractRepair, CapabilityPromotion, SensorRecovery, DegradedValidationStop, NodeSplit, InterfaceInsertion, SubgraphReplan, UserEscalation
`EscalationReport`	node_id, category, action, energy_snapshot, stage_outcomes, evidence

Sheaf Validation:

Type	Description
`SheafValidatorClass` (7 variants)	ExportImportConsistency, DependencyGraphConsistency, SchemaContractCompatibility, BuildGraphConsistency, TestOwnershipConsistency, CrossLanguageBoundary, PolicyInvariantConsistency
`SheafValidationResult`	validator_class, plugin_source, passed, evidence_summary, v_sheaf_contribution, requeue_targets

Provisional Branches:

Type	Description
`ProvisionalBranch`	branch_id, node_id, state, parent_seal_hash, sandbox_dir
`ProvisionalBranchState`	Active → Sealed → Merged / Flushed
`InterfaceSealRecord`	node_id, sealed_path, artifact_kind, seal_hash, version
`BranchFlushRecord`	parent_node_id, flushed_branch_ids, requeue_node_ids, reason
`BlockedDependency`	child_node_id, parent_node_id, required_seal_paths

Structural Digests:

Type	Description
`StructuralDigest`	Content hash of Interface artifacts (signatures, schemas, seals)
`SummaryDigest`	Compressed summaries (IntentSummary, VerifierResults, DesignRationale)
`ArtifactKind`	Signature, Schema, SymbolInventory, InterfaceSeal

Planning and Budget:

Type	Description
`BudgetEnvelope`	Session-level budget caps: max_steps, max_revisions, max_cost_usd, usage counters
`FeatureCharter`	Scope governance: max_modules, max_files, max_revisions, language_constraint
`PlanRevision`	Versioned plan: revision_id, sequence, plan, reason, supersedes, status (Active/Superseded/Cancelled)
`RepairFootprint`	Correction audit: affected_files, applied_bundle, diagnosis, resolved flag
`PlanningPolicy` (5 variants)	Adaptive agent gating: LocalEdit (skip architect), FeatureIncrement (default), LargeFeature, GreenfieldBuild, ArchitecturalRevision. Methods: `needs_architect()`, `needs_speculator()`

12.1.10 Events System

The event system uses unbounded tokio channels:

```
// In perspt_core::events::channel
pub type EventSender = UnboundedSender<AgentEvent>;
pub type EventReceiver = UnboundedReceiver<AgentEvent>;
pub type ActionSender = UnboundedSender<AgentAction>;
pub type ActionReceiver = UnboundedReceiver<AgentAction>;
```

`AgentEvent` has ~35 variants covering the full PSP-5 lifecycle:

- **Planning**: `PlanReady`, `PlanGenerated`, `PlanRevised`
- **Execution**: `NodeSelected`, `BundleApplied`, `NodeCompleted`
- **Verification**: `VerificationComplete`, `DegradedVerification`, `SensorFallback`
- **Sheaf**: `SheafValidationComplete`
- **Branches**: `BranchCreated`, `InterfaceSealed`, `BranchFlushed`, `BranchMerged`
- **Escalation**: `EscalationClassified`, `GraphRewriteApplied`
- **Context**: `ContextDegraded`, `ContextBlocked`, `ProvenanceDrift`
- **Budget**: `BudgetUpdated`
- **File Ops**: `FileDeleted`, `FileMoved`
- **UI**: `ApprovalRequest`, `TaskStatusChanged`, `EnergyUpdated`, `Log`
- **Lifecycle**: `Complete`, `Error`, `ModelFallback`, `ToolReadiness`

12.1.11 Data Flow

```
User Input
    |
[perspt-cli]  Parse args (clap)
    |
[perspt-core]  Config + Provider init
```

(continues on next page)

(continued from previous page)

```
    |
+---+---+
|       |
chat    agent
|       |
[tui]   [perspt-agent]
|       |
|       +-- SRBNOrchestrator
|       |     +-- detect_workspace()  -> [plugins]
|       |     +-- plan_task()         -> [Architect Agent]
|       |     +-- execute_dag()       -> [Actuator Agent]
|       |     +-- verify_node()       -> [LSP, TestRunner, Verifier Agent]
|       |     +-- sheaf_validate()    -> [Sheaf Validators]
|       |     +-- commit_node()       -> [MerkleLedger]
|       |     +-- derive SessionOutcome from completed/escalated counts
|       |     +-- emit Complete event with outcome
|       |
|       +-- AgentTools  (filesystem, search, commands)
|       +-- LspClient   (JSON-RPC stdio)
|       +-- TestRunner   (plugin-driven)
|       +-- ContextRetriever (workspace search)
|       |
|       +-- EventSender --> [perspt-tui AgentApp]
|       +-- ActionReceiver <-- [perspt-tui ReviewModal]
|
[perspt-store]  DuckDB persistence
[perspt-policy]  Starlark rule evaluation
[perspt-sandbox]  Process isolation
```

12.1.12 Streaming Contract

Both chat and agent mode use the same streaming protocol:

1. LLM requests stream chunks over `mpsc::UnboundedSender<String>`
2. End-of-response signaled by `EOT_SIGNAL` (`<|EOT|>`)
3. Provider sends EOT — UI never adds its own
4. UI batches channel messages, handles first EOT, ignores duplicates
5. Streaming buffer updates the last assistant message live
6. Pending inputs queue until EOT is received

> **Warning**
>
> Never block the UI select loop. Spawn LLM work on tokio tasks and send results via the channel.

12.2 Contributing

12.2.1 Development Setup

```
# Clone
git clone https://github.com/eonseed/perspt.git
```

(continues on next page)

(continued from previous page)

```
cd perspt

# Build
cargo build

# Run tests
cargo test

# Lint
cargo clippy -- -D warnings

# Format check
cargo fmt -- --check
```

12.2.2 Project Structure

```
crates/
  perspt-core/     # Types, config, LLM, events, plugins
  perspt-agent/    # Orchestrator, agents, ledger, tools
  perspt-tui/      # Chat + Agent TUI
  perspt-cli/      # CLI entry (clap)
  perspt-store/    # DuckDB persistence
  perspt-policy/   # Starlark policies
  perspt-sandbox/  # Command sandboxing
tests/             # Integration tests
docs/              # Sphinx documentation
```

12.2.3 Coding Standards

1. **Clippy clean** — `cargo clippy -- -D warnings` must pass
2. **Formatted** — `cargo fmt` with default settings
3. **Tests pass** — `cargo test` must pass all tests
4. **No "println¡' in UI paths** — Use the event system or `env_logger`
5. **Error types** — Use `ErrorType` enum for categorized errors
6. **Streaming safety** — Never block the UI select loop; spawn on tokio tasks

12.2.4 Commit Messages

- Describe what changed, not the sequence
- Do NOT include phase numbers or commit sequence numbers
- Keep the subject line under 72 characters

```
# Good
Add sheaf validation for cross-language boundaries

# Bad
Commit 3/7: Phase 2 - Add sheaf validation
```

12.2.5 PR Workflow

1. Create a feature branch from `main`
2. Make changes with passing tests
3. Run the full check suite:

```
cargo build && cargo test && cargo clippy -- -D warnings && cargo fmt -- --check
```

4. Push and open a PR
5. Address review feedback

12.2.6 Documentation

Documentation uses Sphinx with reStructuredText:

```
# Build HTML docs
cd docs/perspt_book && uv run make html

# Build PDF docs
cd docs/perspt_book && uv run make latexpdf

# Live preview
cd docs/perspt_book && uv run sphinx-autobuild source build/html
```

See the `Generate Documentation` and `Build Sphinx HTML Documentation` VS Code tasks for convenience.

12.3 Extending Perspt

12.3.1 Adding a Language Plugin

Language plugins implement the `LanguagePlugin` trait from `perspt-core`.

1. Create a new module in `crates/perspt-core/src/plugin/`:

```
pub struct GoPlugin;

impl LanguagePlugin for GoPlugin {
    fn name(&self) -> &str { "go" }
    fn extensions(&self) -> &[&str] { &["go"] }
    fn key_files(&self) -> &[&str] { &["go.mod", "go.sum"] }

    fn detect(&self, path: &Path) -> bool {
        path.join("go.mod").exists()
    }

    fn get_init_action(&self, opts: &InitOptions) -> ProjectAction {
        ProjectAction::ExecCommand {
            command: format!("go mod init {}", opts.name),
            description: "Initialize Go module".into(),
        }
    }

    fn test_command(&self) -> Option<String> {
        Some("go test ./...".into())
    }
```

(continues on next page)

(continued from previous page)

```
    fn syntax_check_command(&self) -> Option<String> {
        Some("go vet ./...".into())
    }

    fn verifier_profile(&self) -> VerifierProfile {
        // Define capabilities for each verifier stage
        // with primary and fallback commands
    }
}
```

2. Register in the `PluginRegistry`
3. Add LSP config for `gopls`

12.3.2 Adding an Agent Tool

Tools are defined in `crates/perspt-agent/src/tools.rs`:

```
impl AgentTools {
    pub fn get_available_tools(&self) -> Vec<ToolDefinition> {
        vec![
            // ... existing tools ...
            ToolDefinition {
                name: "my_tool".into(),
                description: "Description".into(),
                parameters: vec![
                    ToolParameter {
                        name: "arg1".into(),
                        description: "First argument".into(),
                        required: true,
                    },
                ],
            },
        ]
    }

    async fn execute_my_tool(&self, args: &HashMap<String, String>)
        -> ToolResult
    {
        // Implementation
    }
}
```

12.3.3 Adding a Sheaf Validator

Sheaf validators check cross-node contracts. Add a new variant to `SheafValidatorClass` in `perspt-core/src/types.rs`:

```
pub enum SheafValidatorClass {
    ExportImportConsistency,
    DependencyGraphConsistency,
    // ... existing variants ...
```

(continues on next page)

(continued from previous page)

```
    MyNewValidator,     // Add new variant
}
```

Then implement the validation logic in the orchestrator's sheaf validation phase.

12.3.4 Adding an LLM Provider

The `genai` adapter in `perspt-core/src/llm_provider.rs` handles providers. To add a new provider:

1. Add the adapter kind to `str_to_adapter_kind()`
2. Map the env var in `new_with_config()`
3. Add default model fallbacks in `perspt-cli/src/main.rs`
4. Update the auto-detection priority in config

```
fn str_to_adapter_kind(provider: &str) -> AdapterKind {
    match provider {
        "openai" => AdapterKind::OpenAI,
        "anthropic" => AdapterKind::Anthropic,
        // ... existing providers ...
        "newprovider" => AdapterKind::NewProvider,
        _ => AdapterKind::OpenAI,
    }
}
```

12.3.5 Adding a Starlark Policy

Create a `.star` file in the policy directory (`~/.config/perspt/policies/`):

```
# custom_policy.star

def check_file_write(path, content):
    # Called before any file write.
    if path.endswith(".env"):
        return deny("Cannot write .env files")
    return allow()

def check_command(cmd):
    # Called before any command execution.
    if "curl" in cmd and "http://" in cmd:
        return prompt("Insecure HTTP request: " + cmd)
    return allow()
```

The `PolicyEngine` loads all `.star` files from the policy directory automatically.

12.4 Testing

12.4.1 Test Infrastructure

Perspt uses Rust's standard test framework with `#[tokio::test]` for async tests.

```
# Run all tests
cargo test
```

(continues on next page)

(continued from previous page)

```
# Run tests for a specific crate
cargo test -p perspt-core
cargo test -p perspt-agent
cargo test -p perspt-store

# Run a specific test
cargo test test_name

# Run with output
cargo test -- --nocapture
```

12.4.2 Test Organization

Location	Type	Description
`crates/*/src/*.rs`	Unit tests	`#[cfg(test)] mod tests` blocks
`tests/`	Integration tests	Cross-crate integration
`crates/perspt-store/`	Store tests	DuckDB schema, CRUD, ledger

12.4.3 Test Patterns

In-Memory Store:

Use `SessionStore::in_memory()` for tests that need persistence without disk I/O:

```rust
#[tokio::test]
async fn test_ledger_commit() {
    let store = SessionStore::in_memory().unwrap();
    let ledger = MerkleLedger::from_store(store);
    // ... test ledger operations
}
```

Plugin Testing:

Test plugin detection and verifier profiles:

```rust
#[test]
fn test_python_plugin_detection() {
    let plugin = PythonPlugin;
    let dir = tempdir().unwrap();
    std::fs::write(dir.path().join("pyproject.toml"), "").unwrap();
    assert!(plugin.detect(dir.path()));
}
```

Energy Computation:

Test energy calculation with known inputs:

```rust
#[test]
fn test_energy_total() {
    let energy = EnergyComponents {
        v_syn: 1.0,
```

(continues on next page)

(continued from previous page)

```
        v_str: 0.0,
        v_log: 2.0,
        v_boot: 0.0,
        v_sheaf: 0.0,
    };
    let contract = BehavioralContract {
        energy_weights: (1.0, 0.5, 1.0),
        ..Default::default()
    };
    assert_eq!(energy.total(&contract), 3.0);
}
```

Event Testing:

Test event channel communication:

```
#[tokio::test]
async fn test_event_flow() {
    let (tx, mut rx) = event_channel();
    tx.send(AgentEvent::Log { message: "test".into() }).unwrap();
    let event = rx.recv().await.unwrap();
    assert!(matches!(event, AgentEvent::Log { .. }));
}
```

12.4.4 Quality Gates

All PRs must pass:

```
cargo build                    # Compile
cargo test                     # All tests
cargo clippy -- -D warnings    # No warnings
cargo fmt -- --check           # Formatted
```

The project currently has 239+ tests across all crates.

12.4.5 Panic Safety

`main` installs a panic hook that:

1. Restores terminal (raw mode off, leave alternate screen)
2. Prints the panic message with guidance
3. Exits cleanly

Test this with `tests/panic_handling_test.rs`.

Chapter 13

Changelog

13.1 Version 0.6.0 — “kukuza”

Nurturing the foundation, empowering the core.

Workspace-Wide Dependency Upgrades:

- **duckdb Upgrade** — Bumped `duckdb` requirement to `=1.10503.1` in the workspace root.
- **askama Upgrade** — Bumped `askama` requirement to `0.16` in `perspt-dashboard`.
- **diffy Upgrade** — Bumped `diffy` requirement to `0.5` in `perspt-agent`.
- **genai Upgrade** — Bumped `genai` requirement to `0.6.1` in `perspt-core`.
- **starlark Upgrade** — Bumped `starlark` requirement to `0.14` in `perspt-policy`.

API Alignments & Adaptations:

- **genai API Alignment** — Updated `all_model_names` inside `llm_provider.rs` to pass `()` for the new `ProviderConfig` parameter.
- **starlark API Alignment** — Adapted policy loading in `engine.rs` to use `Module::with_temp_heap` since `Module::new` was deprecated and made private in `0.14`.

Specification & Ecosystem Maintenance:

- **PSP-7 Finalization** — Formally transitioned the PSP-7 specification to `Final` status under the PSP-000001 process.

13.2 Version 0.5.9 — “□□□□”

Perfecting the essence until the work needs no words to shine.

PSP-7: Robust Correction Loop Contracts:

- **Structured Artifact Bundle Format** — Switched correction prompt from free-form `File: ...` output to a strict JSON `{ artifacts: [], commands: [] }` schema. Includes exact target paths from evidence so the LLM targets the correct files, reducing parse failures.
- **AgentTools Integration** — Routed correction commands through `execute_correction_command()` to integrate with plugin policy, user approval gates, and tool failure tracking.
- **Typed Parse Pipeline** — Replaced Option-based bundle extraction with a 5-layer fail-closed typed parse pipeline. Added `RetryClassification` (Retarget, MalformedRetry, SupportFileViolation, Replan) population in `CorrectionAttemptRecord`.
- **Manifest Policy Enforcement** — Added semantic validation to prevent implicit mutation of root manifests (Cargo.toml, package.json) unless explicitly listed as output targets, while preserving legal support files.

- **Strict Budget Exhaustion** — Widened budget exhaustion checks from cost-only to `any_exhausted()` to properly respect step and revision caps before attempting LLM calls.

LLM Provider Maintenance:

- **genai Upgrade** — Bumped `genai` dependency from 0.5.1 to 0.5.3 (stable patch release with bug fixes).
- **Dead Code Cleanup** — Removed `generate_response_with_history()` and `generate_response_with_options()` which had zero callers across the workspace and were the only methods leaking `genai::ChatOptions` into the public API surface.
- **Clippy Fixes** — Fixed *clippy::unnecessary-sort-by* and applied *clippy::collapsible-match* auto-fixes for Rust 1.95 compatibility.

13.3 Version 0.5.8 — "Qualitätsveredelung"

Orchestration State Overhaul Release

> "Qualitätsveredelung — the craft of refining what exists until its quality speaks for itself. Not new features, but the quiet discipline of making every state transition truthful, every metric honest, and every dead path removed."

Orchestration Correctness (Refs: #112, #113, #114, #116):

- **SessionOutcome enum** — New `SessionOutcome` type (Success, PartialSuccess, Failed) derived from actual completed/escalated node counts. The `Complete` event now carries truthful outcomes instead of unconditional `success: true`.
- **NodeOutcome enum** — `execute_node()` returns `Result<NodeOutcome>` where `NodeOutcome` is `Completed` or `Escalated`, replacing the previous `Result<()>` that could not distinguish outcomes.
- **Correct session outcome derivation** — `run_orchestration()` and `run_resumed_inner()` track completed/escalated counts per node and derive the final `SessionOutcome` accordingly.
- **Always-on LLM telemetry** — `call_llm_with_logging()` now records token usage (in/out), latency, and estimated cost via `record_llm_usage()` after every LLM call, regardless of `--log-llm`. The flag now only controls verbose prompt/response text persistence.
- **Budget envelope persistence** — `upsert_budget_envelope()` called after each `BudgetUpdated` event to persist cost/step tracking to the database.
- **Sandbox-aware context retrieval** — `ContextRetriever` in `step_speculate()` uses `effective_working_dir(idx)` (the node's sandbox directory) instead of the workspace root. Sandbox file tree listings included in actuator and correction prompts for better generation grounding.

Type-Safe State Management (Refs: #114):

- **NodeState::from_display_str()** — Case-insensitive canonical parser with legacy aliases ("running" → Coding, "stable" → Completed, "retrying" → Retry). Replaces all ad-hoc string parsing across the codebase.
- **NodeState helpers** — `is_success()` (true only for Completed), `is_active()` (true for Coding, Verifying, Planning, Retry, SheafCheck, Committing), and `Display` impl producing lowercase labels.
- **CLI state cleanup** — All string-based state comparisons in `status.rs`, `agent.rs`, and `resume.rs` replaced with `NodeState::from_display_str()` and type-safe helper methods.

Dead Code Elimination:

- Removed 16 unused functions across `perspt-core`, `perspt-store`, `perspt-agent`, `perspt-policy`, `perspt-tui` (~234 lines)
- Downgraded `canonicalize` to `pub(crate)` in `perspt-policy`
- Removed orphaned `sha2` dependency from `perspt-store`

Bug Fixes (Refs: #107, #111):

- **Session status stuck at RUNNING** — Status now persisted in `end_session()` with `COALESCE`-based finalization guarantee (#111)

- **LLM token counts always zero** — Real provider token usage (prompt + completion tokens) extracted from `genai` `ChatResponse::usage` and persisted per request (#107, #110)

Tests:

- 7 new tests covering `NodeState` parsing round-trips, `SessionOutcome` equality, `NodeOutcome` discriminants, and session outcome derivation from completed/escalated counts
- Total test count: 359 (up from 352)

Documentation:

- Updated README: fixed crate count (8 → 9), deduplicated dashboard command, added missing agent flags (`--single-file`, `--verifier-strictness`, `--output-plan`, all `--*-fallback-model`), aligned contributing commands with CI gates
- Updated Perspt Book: SRBN architecture (Phase 7 → Commit & Outcome), CLI reference (`logs` always shows token metrics), developer architecture guide (orchestrator lifecycle, NodeOutcome, SessionOutcome in type inventory and data flow), workspace crates (removed dead `is_safe_for_auto_exec`)
- Updated PSP-5: execution flow steps 8–11 with Completed/Escalated paths and SessionOutcome derivation, headless output with `OUTCOME` line, added Orchestration State Overhaul implementation appendix

13.4 Version 0.5.7 — "navikaran □□□□□□"

Dashboard UX Polish Release

> "Bridging the purpose of Ikigai with the momentum of Kaizen — renewal through continuous, intentional refinement."

Dashboard UX Improvements (PSP-6 continued):

- **Custom DaisyUI 5 themes** — `perspt-light` and `perspt-dark` themes with orange/pink oklch palette (WCAG AA compliant), powered by `@plugin "daisyui/theme"` blocks
- **Theme toggle** — Navbar button with sun/moon icons, localStorage persistence, and migration from legacy theme names
- **Friendly session names** — Deterministic human-readable names (e.g. "bold-hawk") derived from session UUIDs via hash-indexed adjective+noun arrays
- **Breadcrumb friendly names** — All six session sub-pages show friendly name with UUID-on-hover tooltip
- **Session card layout** — Stacked vertical cards with `btn-outline` sub-page buttons replacing ghost buttons
- **Task text formatting** — `whitespace-pre-line` rendering for readable multi-line task descriptions
- **Collapse arrow fix** — `pe-10` padding on DAG and LLM collapse summaries to prevent arrow overlap with text
- **Decisions page resilience** — All six store queries use `unwrap_or_default()` instead of `?` early-return, preventing 503 errors on partial data
- **Paginated overview** — 20 sessions per page with DaisyUI `join` pagination controls, backed by `list_sessions_paginated()` and `count_sessions()` store methods
- **Login page theme** — Updated to `perspt-light` default

CI & Build:

- **Node.js in CI** — Added `actions/setup-node@v4` (Node 22) to CI test matrix and release workflows so `npx @tailwindcss/cli` runs on all runners

Store:

- `list_sessions_paginated(limit, offset)` — LIMIT/OFFSET SQL for paginated session listing
- `count_sessions()` — Total session count for pagination controls

13.5 Version 0.5.6 — "ikigai □□□□"

SRBN Sandbox Revision Flow Release

> "A reason for being — the happiness of always being busy with what you love."

Real-Time Web Dashboard (PSP-6):

- **perspt-dashboard crate** — Axum 0.8 + Askama 0.15 + HTMX 2 + Tailwind v4/DaisyUI 5 web interface for monitoring agent execution
- **Read-only store access** — `SessionStore::open_read_only()` with DuckDB `AccessMode::ReadOnly` for safe concurrent reads alongside the agent
- **Six monitoring pages** — Overview (sessions), DAG (task graph), Energy (Lyapunov components), LLM (request telemetry), Sandbox (provisional branches), Decisions (escalations, sheaf validations, rewrites, plan revisions, repairs, verifications)
- **SSE live updates** — Server-Sent Events stream node statistics every 2 seconds
- **Password authentication** — Random token, HttpOnly/SameSite cookie, Secure flag on non-localhost deployments
- **"perspt dashboard" CLI command** — Launches the dashboard server on a configurable port
- **12 integration tests** — Route smoke tests, SSE content-type, auth flow
- **Store extensions** — `get_session_energy_history()`, `get_all_sheaf_validations()`, `get_all_repair_footprints()`

SRBN Sandbox Revision Flow (PSP-5 Phases 3-12):

- **PlanningPolicy** — Adaptive agent gating with 5 policies (LocalEdit, FeatureIncrement, LargeFeature, GreenfieldBuild, ArchitecturalRevision). `needs_architect()` and `needs_speculator()` gate agent tier activation
- **FeatureCharter auto-creation** — Policy-derived file/module/revision limits created before architect planning so the plan gate has bounds to enforce
- **Speculator lookahead gating** — Speculator tier only activates for LargeFeature, GreenfieldBuild, and ArchitecturalRevision policies
- **BudgetEnvelope session restore** — Step/cost/revision caps restored from DB during `resume` so interrupted sessions honour the original limits
- **Bundle path normalization** — `filter_bundle_to_declared_paths` uses `normalize_artifact_path` for correct comparison of path variants (e.g. `./src/main.rs` vs `src/main.rs`)
- **NodeState::Superseded** — New terminal state for plan amendment (Phase 14 preparation). Updated `is_terminal()`, `parse_node_state()`, and `NodeStatus` conversion
- **Orchestrator module extraction** — `orchestrator.rs` split into 9 submodules: `mod.rs`, `bundle.rs`, `commit.rs`, `convergence.rs`, `init.rs`, `planning.rs`, `repair.rs`, `solo.rs`, `verification.rs`
- **Centralized prompts** — All 15 agent prompts consolidated in `prompts.rs` with constants and `render_*` helpers; duplicates removed from `agent.rs`
- **RepairFootprint-backed correction** — `build_correction_prompt` uses `RepairFootprint` for precise, grounded repair context
- **Greenfield bootstrap ordering** — Plugin-driven project initialization with correct pre-sheafify plugin re-detection
- **Provisional branch lifecycle** — Sandbox-first execution with branch creation, merge, and flush cascade
- **Escalation classification** — 5 categories with 9 rewrite actions and graph surgery support

Documentation:

- Updated architecture docs with PlanningPolicy, FeatureCharter, and NodeState::Superseded documentation
- Added Planning Policy and Feature Charter sections to SRBN architecture guide
- Updated workspace crates docs with orchestrator submodule structure
- Added speculator lookahead and budget restore documentation to agent mode guide
- Fixed energy weight default (gamma=2.0) in advanced features guide
- Refreshed all version references to 0.5.6

13.6 Version 0.5.5

PSP-5 Cross-Platform and CI Stabilization Release

Cross-Platform Fixes:

- **Windows sandbox path normalization** — `list_sandbox_files` now returns forward-slash-separated relative paths on all platforms
- **Windows workspace-bound validation** — `validate_workspace_bound` correctly detects absolute paths with Windows drive prefixes (`C:\...`) and normalizes backslash path separators before POSIX shell tokenization
- **Clippy and fmt CI compliance** — Resolved `items_after_test_module` and `useless_vec` warnings that were failing CI on all platforms

Build and CI Improvements:

- **Removed accidental eval workspace member** — `.perspt-eval/rust_cli` removed from workspace members; `.perspt-eval/` added to `.gitignore`
- **Stabilized cargo doc** — Added `doc = false` to CLI bin target to prevent output collision with the `perspt` library crate
- **Removed deprecated atty dependency** — Replaced with `std::io::IsTerminal` for TTY detection
- **Lockfile refresh** — Cleared hard `cargo audit` vulnerability failures via dependency updates

Documentation:

- Updated workspace coding instructions to match current multi-crate architecture
- Refreshed all version references across the Perspt Book and Sphinx configuration

13.7 Version 0.5.4

PSP-5 Compliance Release

SRBN Agent Enhancements:

- **Per-node error recovery** — Retry logic respects `ErrorType` classification (`Compilation`, `ToolFailure`, `ReviewRejection`) with separate counters
- **Multi-file extraction** — Actuator reliably extracts artifact bundles with multiple files from LLM responses
- **Multi-artifact bundles** — Bundle protocol correctly handles write, diff, and command artifacts in a single node
- **Plugin-driven project initialization** — All plugins use `uv init --lib` (Python), `cargo init` (Rust), `npm init` (JS) for proper project scaffolding
- **Degraded verification mode** — When tool binaries are missing, falls back to heuristic verification with clear warnings
- **Sheaf validation** — 7 validator classes for cross-node contract verification

Bug Fixes:

- Fixed `uv init` to `uv init --lib` in `plugin.rs` (2 locations) and `test_runner.rs` (1 location) for correct `src/` layout with `[build-system]`
- Removed dead `test_check_workspace_requirement` test from `orchestrator.rs`

Documentation:

- Complete rewrite of the Perspt Book for PSP-5 accuracy
- Updated developer guide with full type inventory and architecture diagrams
- Added tutorials for headless mode, Python ETL, Rust CLI, and scientific computing
- Updated all model names and provider defaults to current versions

13.8 Version 0.5.3

- SRBN energy convergence improvements
- Provisional branch lifecycle management
- Interface seal and flush cascade support
- DuckDB migration from SQLite

13.9 Version 0.5.2

- Initial SRBN orchestrator implementation
- Per-tier model selection (`--architect-model`, etc.)
- Merkle ledger with DuckDB backend
- Plugin system for Python, Rust, and JavaScript
- Agent TUI with dashboard, task tree, and review modal
- Starlark policy engine
- Basic sandbox command execution
- 10 CLI subcommands (chat, agent, simple-chat, init, config, ledger, status, abort, resume, logs)

13.10 Version 0.5.1

- Multi-provider chat support
- TUI markdown rendering with code blocks
- Simple CLI mode for scripting
- Streaming response protocol with EOT signal

13.11 Version 0.5.0

- Initial release
- Basic chat interface with OpenAI
- Configuration auto-detection from environment

Chapter 14

License

Perspt is released under the GNU Lesser General Public License v3.0 (LGPL v3).

14.1 LGPL v3 License

14.2 What This Means

The LGPL v3 is a copyleft license that provides strong protection for software freedom while allowing linking with proprietary software. Here's what it means in practical terms:

✔ **What you CAN do:**

- **Use** Perspt for any purpose, including commercial projects
- **Modify** the source code to fit your needs
- **Distribute** copies of Perspt
- **Link** Perspt as a library in proprietary software
- **Combine** Perspt with software under different licenses
- **Create** derivative works based on Perspt

! **What you MUST do:**

- **Provide source code** for any modifications to Perspt itself
- **Include** the LGPL v3 license text with distributions
- **Preserve** copyright notices and license information
- **Allow users** to replace the Perspt library with modified versions
- **Make modified source** available under LGPL v3 terms

🚫 **What we DON'T provide:**

- **Warranty** - The software is provided "as is"
- **Liability coverage** - We're not responsible for any damages
- **Support guarantees** - While we strive to help, support is provided on a best-effort basis

14.3 Third-Party Licenses

Perspt depends on several open source libraries, each with their own licenses:

14.3.1 Core Dependencies

Crate	License	Description
tokio	MIT	Async runtime for Rust
ratatui	MIT	Terminal user interface library
serde	MIT/Apache-2.0	Serialization framework
clap	MIT/Apache-2.0	Command line argument parser
anyhow	MIT/Apache-2.0	Error handling library
thiserror	MIT/Apache-2.0	Error derive macros

14.3.2 LLM Integration

Crate	License	Description
genai	MIT/Apache-2.0	Unified LLM provider interface
reqwest	MIT/Apache-2.0	HTTP client library

14.3.3 Terminal and UI

Crate	License	Description
crossterm	MIT	Cross-platform terminal library
unicode-width	MIT/Apache-2.0	Unicode character width calculation
textwrap	MIT	Text wrapping and formatting

14.3.4 Development Dependencies

Crate	License	Description
criterion	MIT/Apache-2.0	Benchmarking library
mockall	MIT/Apache-2.0	Mock object library
tempfile	MIT/Apache-2.0	Temporary file management

14.4 License Compatibility

The LGPL v3 is compatible with most other open source licenses:

Compatible Licenses: - Apache License 2.0 - BSD licenses (2-clause, 3-clause) - ISC License - MIT License - Public Domain (CC0) - GPL v3+ (can be upgraded to GPL)

Special Considerations: - GPL v2: Not directly compatible due to version differences - Proprietary licenses: Can link with LGPL libraries but must allow library replacement - Copyleft licenses: LGPL provides weaker copyleft than GPL

14.5 Commercial Use

Perspt can be freely used in commercial projects:

✔ **Allowed Commercial Uses:**

- **Internal tools** - Use Perspt as part of your development workflow
- **Linked libraries** - Link Perspt as a library in commercial software
- **Service offerings** - Provide Perspt as part of consulting or hosting services
- **Modified library versions** - Create modified versions for internal use
- **Enterprise solutions** - Build enterprise tools that use Perspt

🏷 **Requirements for Commercial Use:**

1. **Include LGPL license text** in your distribution
2. **Maintain copyright notices** from the original code
3. **Provide source code** for any modifications to Perspt itself
4. **Allow library replacement** - users must be able to replace the Perspt library
5. **No trademark usage** without permission (see below)

No additional fees, registrations, or permissions are required.

14.6 Trademark Policy

While the source code is LGPL v3 licensed, trademarks are handled separately:

"Perspt" Name and Logo: - The name "Perspt" and any associated logos are trademarks - You may use the name in accurately describing the software - Commercial use of the name/logo as your own brand requires permission - Modified versions should use different names to avoid confusion

Acceptable Uses: - "Built with Perspt" - "Based on Perspt" - "Powered by Perspt" - "Fork of Perspt"

Requires Permission: - Using "Perspt" as your product name - Using Perspt logos in your branding - Implying official endorsement

14.7 Contributing and License

By contributing to Perspt, you agree that:

1. **Your contributions** will be licensed under the same LGPL v3 License
2. **You have the right** to license your contributions under LGPL v3
3. **You understand** that your contributions may be used commercially
4. **You retain copyright** to your contributions while granting broad usage rights

14.7.1 Contributor License Agreement (CLA)

For substantial contributions, we may request a Contributor License Agreement to:

- Ensure you have the right to contribute the code
- Provide legal protection for the project and users
- Allow for potential future license changes if needed
- Clarify the rights and responsibilities of contributors

14.8 License FAQ

Q: Can I use Perspt in my proprietary software? A: Yes, LGPL v3 allows linking with proprietary software. You must provide the library source and allow replacement.

Q: Can I modify Perspt and sell the modified version? A: Yes, but you must provide the source code for your modifications under LGPL v3.

Q: Do I need to open source my modifications? A: Yes, any modifications to Perspt itself must be made available under LGPL v3.

Q: Can I remove the copyright notices? A: No, you must preserve the copyright notices and license information in all copies.

Q: What if I only use parts of the code? A: The LGPL v3 license still applies to any substantial portions you use.

Q: Can I change the license of my derivative work? A: You can license your own code separately, but Perspt parts must remain LGPL v3.

Q: Do I need to attribute Perspt in my application? A: Yes, you must include the LGPL v3 license and copyright notices.

14.9 Getting Legal Advice

This page provides general information about the LGPL v3 License and is not legal advice. For specific legal questions:

- **Consult** with a qualified attorney
- **Review** the full license text carefully
- **Consider** your specific use case and jurisdiction
- **Seek** professional legal counsel for commercial decisions

14.10 Reporting License Issues

If you believe there's a license violation or have questions about licensing:

- **Email**: legal@perspt.dev
- **GitHub Issues**: License Questions
- **Include** specific details about the concern or question

We take licensing seriously and will investigate all reports promptly.

See also

- *Acknowledgments* - Credits and thanks to contributors
- *Contributing* - How to contribute to the project
- GNU Project - Official LGPL v3 License text

Chapter 15

Acknowledgments

Perspt is built on the shoulders of giants. We extend our gratitude to the many open-source projects, libraries, and communities that made this project possible.

15.1 Theoretical Foundation

The SRBN engine at the heart of Perspt's experimental agent mode is based on the paper:

> **"Stability is All You Need: Lyapunov-Guided Hierarchies for Long-Horizon LLM Reliability"** by **Vikrant R.** and **Ronak R.** (pre-publication).

The paper introduces a topological framework that reformulates LLM agency as a sheaf-theoretic control problem, replacing probabilistic search with Lyapunov stability analysis. Key theoretical contributions adopted in Perspt include:

- **Input-to-State Stability (ISS)** — bounded reasoning errors yield bounded system deviation (paper proof)
- **Flow Matching Barriers** — project diverging trajectories back onto the safe manifold (paper result)
- **Adaptive Flow Speculation** — latency reduction via predictive branch execution
- **Logarithmic Reliability Scaling** — the paper predicts $O(\log N)$ retry cost instead of exponential decay

Perspt serves as an experimental testbed for exploring these theoretical ideas in real-world multi-file coding workloads. The mathematical framework is mature; empirical benchmarks on Perspt's implementation have not yet been published.

15.2 Core Dependencies

15.2.1 AI and LLM Integration

genai
The foundation of Perspt's multi-provider support. This exceptional crate provides unified interfaces to multiple AI providers and automatically stays up-to-date with new models and capabilities.

- **Project**: genai
- **License**: MIT/Apache 2.0
- **Impact**: Enables seamless integration with OpenAI, Anthropic, Google, Groq, Cohere, XAI, DeepSeek, and Ollama providers

serde & serde_json
Rust's premier serialization framework, powering Perspt's configuration management and API communication.

- **Project**: serde
- **License**: MIT/Apache 2.0
- **Impact**: JSON configuration parsing, API request/response handling

15.2.2 User Interface and Terminal

ratatui
The modern, feature-rich TUI framework that powers Perspt's interactive terminal interface.
- **Project**: ratatui
- **License**: MIT
- **Impact**: Rich terminal UI, markdown rendering, scrollable chat interface

crossterm
Cross-platform terminal manipulation library enabling consistent behavior across operating systems.
- **Project**: crossterm
- **License**: MIT
- **Impact**: Keyboard input handling, terminal control, cross-platform compatibility

15.2.3 Async Runtime and Concurrency

tokio
The asynchronous runtime that enables Perspt's responsive, non-blocking architecture.
- **Project**: tokio
- **License**: MIT
- **Impact**: Async/await support, concurrent LLM requests, responsive UI

15.2.4 Error Handling and Utilities

anyhow
Elegant error handling that makes Perspt's error messages helpful and actionable.
- **Project**: anyhow
- **License**: MIT/Apache 2.0
- **Impact**: Comprehensive error context, user-friendly error messages

clap
Command-line argument parsing that makes Perspt easy to use and configure.
- **Project**: clap
- **License**: MIT/Apache 2.0
- **Impact**: CLI interface, help generation, argument validation

15.3 Documentation Tools

Sphinx
The documentation generator that created this beautiful book-style documentation.
- **Project**: Sphinx
- **License**: BSD
- **Impact**: Professional documentation, PDF generation, cross-references

Furo Theme
The modern, accessible Sphinx theme that makes this documentation a pleasure to read.
- **Project**: Furo
- **License**: MIT
- **Impact**: Beautiful documentation design, responsive layout, accessibility

15.4 Development Tools

Rust Language
The systems programming language that makes Perspt fast, safe, and reliable.
- **Project**: Rust

- **License**: MIT/Apache 2.0
- **Impact**: Memory safety, performance, excellent tooling ecosystem

cargo
Rust's package manager and build system that makes development smooth and dependency management effortless.

- **Project**: Part of Rust toolchain
- **License**: MIT/Apache 2.0
- **Impact**: Dependency management, build automation, testing framework

15.5 Community and Inspiration

15.5.1 AI Provider Communities

OpenAI
For creating GPT models and establishing many of the patterns that define modern AI interaction.
Anthropic
For Claude models and their pioneering work in AI safety and helpful, harmless, and honest AI systems.
Google
For Gemini models and their contributions to accessible AI technology.
Groq
For ultra-fast inference infrastructure and democratizing AI speed.
Cohere
For enterprise-grade language models and excellent developer tools.
XAI
For Grok models and advancing conversational AI capabilities.
DeepSeek
For their contributions to the open-source AI ecosystem.
Ollama
For making local AI model hosting accessible and user-friendly.

15.5.2 Open Source Ecosystem

GitHub
For providing the platform that enables collaborative development and open-source sharing.
crates.io
Rust's package registry that makes sharing and discovering Rust libraries effortless.
docs.rs
For automatically generating and hosting documentation for Rust crates.

15.5.3 Terminal and CLI Inspiration

The terminal and CLI interface draws inspiration from many excellent tools:

- **htop** - For showing how terminal UIs can be both beautiful and functional
- **tmux** - For terminal multiplexing concepts and keyboard navigation patterns
- **vim/neovim** - For modal editing concepts and efficient keyboard shortcuts
- **fzf** - For demonstrating responsive, interactive terminal interfaces

15.5.4 Rust Community Projects

Many patterns and approaches in Perspt were learned from studying excellent Rust projects:

- **ripgrep** - For performance optimization and user experience design
- **bat** - For beautiful terminal output and syntax highlighting
- **exa/eza** - For modern CLI design and colored output

- **gitui** - For TUI application architecture and event handling

15.6 Testing and Quality Assurance

Users and Beta Testers
The early adopters and users who provided feedback, reported bugs, and suggested improvements.
Security Researchers
For responsible disclosure of security issues and helping make Perspt more secure.
Documentation Reviewers
For helping improve the clarity and completeness of this documentation.

15.7 Special Thanks

AI Safety Research Community
For ongoing work to make AI systems more reliable, interpretable, and aligned with human values.
Open Source Contributors
To everyone who contributes to open-source projects, from major features to documentation fixes.
Rust Community
For creating and maintaining an inclusive, helpful community that makes Rust development a joy.
Terminal Enthusiasts
For keeping the art of terminal-based applications alive and pushing the boundaries of what's possible in text-based interfaces.

15.8 Contributing Back

Perspt aims to be a good citizen of the open-source ecosystem. We contribute back by:

Open Source Release
Perspt itself is released under the LGPL v3 license, allowing anyone to use, modify, and distribute it.
Documentation Standards
This comprehensive documentation serves as an example of thorough project documentation.
Best Practices Sharing
Through blog posts, talks, and code examples, we share what we've learned building Perspt.
Upstream Contributions
When we find bugs or missing features in dependencies, we contribute fixes and improvements back to those projects.

15.9 License Information

Perspt is licensed under the LGPL v3 License. For complete license information, see *License*.

All dependencies are used in accordance with their respective licenses. We are grateful to all the authors and maintainers who choose to share their work under permissive open-source licenses.

15.10 Get Involved

Want to contribute to Perspt or the broader ecosystem?

Report Issues
Help improve Perspt by reporting bugs, suggesting features, or improving documentation.
Contribute Code
See our *Contributing* guide for how to contribute code improvements.

Share Knowledge
Write blog posts, create tutorials, or give talks about your experience with Perspt.
Support Dependencies
Consider contributing to the open-source projects that Perspt depends on.
Spread the Word
Help others discover Perspt and the amazing ecosystem of Rust and AI tools.

—

Thank you to everyone who makes open-source software development possible. Your contributions, large and small, make projects like Perspt possible.

Chapter 16

Indices

- genindex
- modindex
- search